

PART 3: CRYPTOGRAPHIC DATA INTEGRITY ALGORITHMS

CHAPTER 11

CRYPTOGRAPHIC HASH FUNCTIONS

11.1 Applications of Cryptographic Hash Functions

- Message Authentication

- Digital Signatures

- Other Applications

11.2 Two Simple Hash Functions

11.3 Requirements and Security

- Security Requirements for Cryptographic Hash Functions

- Brute-Force Attacks

- Cryptanalysis

11.4 Hash Functions Based on Cipher Block Chaining

11.5 Secure Hash Algorithm (SHA)

- SHA-512 Logic

- SHA-512 Round Function

- Example

11.6 SHA-3

11.7 Recommended Reading and Web Sites

11.8 Key Terms, Review Questions, and Problems

Appendix 11A Mathematical Basis of the Birthday Attack

Each of the messages, like each one he had ever read of Stern's commands, began with a number and ended with a number or row of numbers. No efforts on the part of Mungo or any of his experts had been able to break Stern's code, nor was there any clue as to what the preliminary number and those ultimate numbers signified.

—Talking to Strange Men, Ruth Rendell

The Douglas Squirrel has a distinctive eating habit. It usually eats pine cones from the bottom end up. Partially eaten cones can indicate the presence of these squirrels if they have been attacked from the bottom first. If, instead, the cone has been eaten from the top end down, it is more likely to have been a crossbill finch that has been doing the dining.

—Squirrels: A Wildlife Handbook, Kim Long

KEY POINTS

- ◆ A hash function maps a variable-length message into a fixed-length hash value, or message digest.
- ◆ Virtually all cryptographic hash functions involve the iterative use of a compression function.
- ◆ The compression function used in secure hash algorithms falls into one of two categories: a function specifically designed for the hash function or an algorithm based on a symmetric block cipher. SHA and Whirlpool are examples of these two approaches, respectively.

A hash function H accepts a variable-length block of data M as input and produces a fixed-size hash value $h = H(M)$. A “good” hash function has the property that the results of applying the function to a large set of inputs will produce outputs that are evenly distributed and apparently random. In general terms, the principal object of a hash function is data integrity. A change to any bit or bits in M results, with high probability, in a change to the hash code.

The kind of hash function needed for security applications is referred to as a **cryptographic hash function**. A cryptographic hash function is an algorithm for which it is computationally infeasible (because no attack is significantly more efficient than brute force) to find either (a) a data object that maps to a pre-specified hash result (the one-way property) or (b) two data objects that map to the same hash result (the collision-free property). Because of these characteristics, hash functions are often used to determine whether or not data has changed.

Figure 11.1 depicts the general operation of a cryptographic hash function. Typically, the input is padded out to an integer multiple of some fixed length (e.g., 1024 bits), and the padding includes the value of the length of the original message in bits. The length field is a security measure to increase the difficulty for an attacker to produce an alternative message with the same hash value.

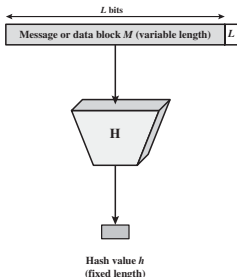


Figure 11.1 Black Diagram of Cryptographic Hash Function; $h = H(M)$

This chapter begins with a discussion of the wide variety of applications for cryptographic hash functions. Next, we look at the security requirements for such functions. Then we look at the use of cipher block chaining to implement a cryptographic hash function. The remainder of the chapter is devoted to the most important and widely used family of cryptographic hash functions, the Secure Hash Algorithm (SHA) family.

Appendix N describes Whirlpool, another popular cryptographic hash function.

11.1 APPLICATIONS OF CRYPTOGRAPHIC HASH FUNCTIONS

Perhaps the most versatile cryptographic algorithm is the cryptographic hash function. It is used in a wide variety of security applications and Internet protocols. To better understand some of the requirements and security implications for cryptographic hash functions, it is useful to look at the range of applications in which it is employed.

Message Authentication

Message authentication is a mechanism or service used to verify the integrity of a message. Message authentication assures that data received are exactly as sent (i.e., contain no modification, insertion, deletion, or replay). In many cases, there is a requirement that the authentication mechanism assures that purported identity of the sender is valid. When a hash function is used to provide message authentication, the hash function value is often referred to as a **message digest**.

Figure 11.2 illustrates a variety of ways in which a hash code can be used to provide message authentication, as follows.

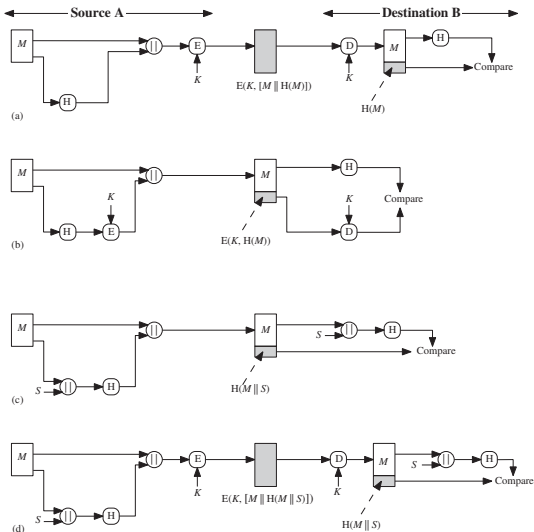


Figure 11.2 Simplified Examples of the Use of a Hash Function for Message Authentication

- The message plus concatenated hash code is encrypted using symmetric encryption. Because only A and B share the secret key, the message must have come from A and has not been altered. The hash code provides the structure or redundancy required to achieve authentication. Because encryption is applied to the entire message plus hash code, confidentiality is also provided.
- Only the hash code is encrypted, using symmetric encryption. This reduces the processing burden for those applications that do not require confidentiality.
- It is possible to use a hash function but no encryption for message authentication. The technique assumes that the two communicating parties share a common secret value S . A computes the hash value over the concatenation of M and S and

appends the resulting hash value to M . Because B possesses S , it can recompute the hash value to verify. Because the secret value itself is not sent, an opponent cannot modify an intercepted message and cannot generate a false message.

- d. Confidentiality can be added to the approach of method (c) by encrypting the entire message plus the hash code.

When confidentiality is not required, method (b) has an advantage over methods (a) and (d), which encrypts the entire message, in that less computation is required. Nevertheless, there has been growing interest in techniques that avoid encryption (Figure 11.2c). Several reasons for this interest are pointed out in [TSUD92].

- Encryption software is relatively slow. Even though the amount of data to be encrypted per message is small, there may be a steady stream of messages into and out of a system.
- Encryption hardware costs are not negligible. Low-cost chip implementations of DES are available, but the cost adds up if all nodes in a network must have this capability.
- Encryption hardware is optimized toward large data sizes. For small blocks of data, a high proportion of the time is spent in initialization/invocation overhead.
- Encryption algorithms may be covered by patents, and there is a cost associated with licensing their use.

More commonly, message authentication is achieved using a **message authentication code (MAC)**, also known as a **keyed hash function**. Typically, MACs are used between two parties that share a secret key to authenticate information exchanged between those parties. A MAC function takes as input a secret key and a data block and produces a hash value, referred to as the MAC. This can then be transmitted with or stored with the protected message. If the integrity of the message needs to be checked, the MAC function can be applied to the message and the result compared with the stored MAC value. An attacker who alters the message will be unable to alter the MAC value without knowledge of the secret key. Note that the verifying party also knows who the sending party is because no one else knows the secret key.

Note that the combination of hashing and encryption results in an overall function that is, in fact, a MAC (Figure 11.2b). That is, $E(K, H(M))$ is a function of a variable-length message M and a secret key K , and it produces a fixed-size output that is secure against an opponent who does not know the secret key. In practice, specific MAC algorithms are designed that are generally more efficient than an encryption algorithm.

We discuss MACs in Chapter 12.

Digital Signatures

Another important application, which is similar to the message authentication application, is the **digital signature**. The operation of the digital signature is similar to that of the MAC. In the case of the digital signature, the hash value of a message is encrypted with a user's private key. Anyone who knows the user's public key can verify the integrity of the message that is associated with the digital signature. In this

case, an attacker who wishes to alter the message would need to know the user's private key. As we shall see in Chapter 14, the implications of digital signatures go beyond just message authentication.

Figure 11.3 illustrates, in a simplified fashion, how a hash code is used to provide a digital signature.

- The hash code is encrypted, using public-key encryption with the sender's private key. As with Figure 11.2b, this provides authentication. It also provides a digital signature, because only the sender could have produced the encrypted hash code. In fact, this is the essence of the digital signature technique.
- If confidentiality as well as a digital signature is desired, then the message plus the private-key-encrypted hash code can be encrypted using a symmetric secret key. This is a common technique.

Other Applications

Hash functions are commonly used to create a **one-way password file**. Chapter 20 explains a scheme in which a hash of a password is stored by an operating system rather than the password itself. Thus, the actual password is not retrievable by a hacker who gains access to the password file. In simple terms, when a user enters a password, the hash of that password is compared to the stored hash value for verification. This approach to password protection is used by most operating systems.

Hash functions can be used for **intrusion detection** and **virus detection**. Store $H(F)$ for each file on a system and secure the hash values (e.g., on a CD-R that is

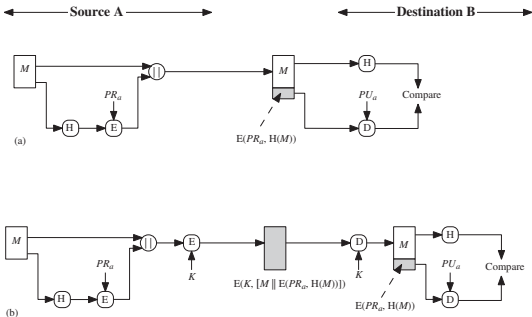


Figure 11.3 Simplified Examples of Digital Signatures

kept secure). One can later determine if a file has been modified by recomputing $H(F)$. An intruder would need to change F without changing $H(F)$.

A cryptographic hash function can be used to construct a **pseudorandom function (PRF)** or a **pseudorandom number generator (PRNG)**. A common application for a hash-based PRF is for the generation of symmetric keys. We discuss this application in Chapter 12.

11.2 TWO SIMPLE HASH FUNCTIONS

To get some feel for the security considerations involved in cryptographic hash functions, we present two simple, insecure hash functions in this section. All hash functions operate using the following general principles. The input (message, file, etc.) is viewed as a sequence of n -bit blocks. The input is processed one block at a time in an iterative fashion to produce an n -bit hash function.

One of the simplest hash functions is the bit-by-bit exclusive-OR (XOR) of every block. This can be expressed as

$$C_i = b_{i1} \oplus b_{i2} \oplus \dots \oplus b_{im}$$

where

- C_i = i th bit of the hash code, $1 \leq i \leq n$
- m = number of n -bit blocks in the input
- b_{ij} = i th bit in j th block
- $\oplus$ = XOR operation

This operation produces a simple parity for each bit position and is known as a longitudinal redundancy check. It is reasonably effective for random data as a data integrity check. Each n -bit hash value is equally likely. Thus, the probability that a data error will result in an unchanged hash value is 2^{-n} . With more predictably formatted data, the function is less effective. For example, in most normal text files, the high-order bit of each octet is always zero. So if a 128-bit hash value is used, instead of an effectiveness of 2^{-128} , the hash function on this type of data has an effectiveness of 2^{-112} .

A simple way to improve matters is to perform a one-bit circular shift, or rotation, on the hash value after each block is processed. The procedure can be summarized as follows.

1. Initially set the n -bit hash value to zero.
2. Process each successive n -bit block of data as follows:
 - a. Rotate the current hash value to the left by one bit.
 - b. XOR the block into the hash value.

This has the effect of “randomizing” the input more completely and overcoming any regularities that appear in the input. Figure 11.4 illustrates these two types of hash functions for 16-bit hash values.

Although the second procedure provides a good measure of data integrity, it is virtually useless for data security when an encrypted hash code is used with a plaintext

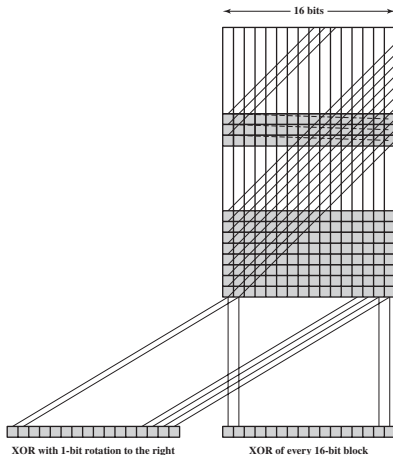


Figure 11.4 Two Simple Hash Functions

message, as in Figures 11.2b and 11.3a. Given a message, it is an easy matter to produce a new message that yields that hash code: Simply prepare the desired alternate message and then append an n -bit block that forces the new message plus block to yield the desired hash code.

Although a simple XOR or rotated XOR (RXOR) is insufficient if only the hash code is encrypted, you may still feel that such a simple function could be useful when the message together with the hash code is encrypted (Figure 11.2a). But you must be careful. A technique originally proposed by the National Bureau of Standards used the simple XOR applied to 64-bit blocks of the message and then an encryption of the entire message that used the cipher block chaining (CBC) mode. We can define the scheme as follows: Given a message M consisting of a sequence of 64-bit blocks $X_1, X_2, \dots, X_N$, define the hash code $h = H(M)$ as the block-by-block XOR of all blocks and append the hash code as the final block:

$$h = X_{N+1} = X_1 \oplus X_2 \oplus \dots \oplus X_N$$

Next, encrypt the entire message plus hash code using CBC mode to produce the encrypted message $Y_1, Y_2, \dots, Y_{N+1}$. [JUEN85] points out several ways in which the ciphertext of this message can be manipulated in such a way that it is not detectable by the hash code. For example, by the definition of CBC (Figure 6.4), we have

$$\begin{aligned}X_1 &= IV \oplus D(K, Y_1) \\X_i &= Y_{i-1} \oplus D(K, Y_i) \\X_{N+1} &= Y_N \oplus D(K, Y_{N+1})\end{aligned}$$

But X_{N+1} is the hash code:

$$\begin{aligned}X_{N+1} &= X_1 \oplus X_2 \oplus \dots \oplus X_N \\&= IV \oplus D(K, Y_1) \oplus [Y_1 \oplus D(K, Y_2)] \oplus \dots \oplus [Y_{N-1} \oplus D(K, Y_N)]\end{aligned}$$

Because the terms in the preceding equation can be XORed in any order, it follows that the hash code would not change if the ciphertext blocks were permuted.

11.3 REQUIREMENTS AND SECURITY

Before proceeding, we need to define two terms. For a hash value $h = H(x)$, we say that x is the **preimage** of h . That is, x is a data block whose hash function, using the function H , is h . Because H is a many-to-one mapping, for any given hash value h , there will in general be multiple preimages. A **collision** occurs if we have $x \neq y$ and $H(x) = H(y)$. Because we are using hash functions for data integrity, collisions are clearly undesirable.

Let us consider how many preimages are there for a given hash value, which is a measure of the number of potential collisions for a given hash value. Suppose the length of the hash code is n bits, and the function H takes as input messages or data blocks of length b bits with $b > n$. Then, the total number of possible messages is 2^b and the total number of possible hash values is 2^n . On average, each hash value corresponds to $2^{b/n}$ preimages. If H tends to uniformly distribute hash values then, in fact, each hash value will have close to $2^{b/n}$ preimages. If we now allow inputs of arbitrary length, not just a fixed length of some number of bits, then the number of preimages per hash value is arbitrarily large. However, the security risks in the use of a hash function are not as severe as they might appear from this analysis. To understand better the security implications of cryptographic hash functions, we need precisely define their security requirements.

Security Requirements for Cryptographic Hash Functions

Table 11.1 lists the generally accepted requirements for a cryptographic hash function. The first three properties are requirements for the practical application of a hash function.

The fourth property, **preimage resistant**, is the one-way property: it is easy to generate a code given a message, but virtually impossible to generate a message given a code. This property is important if the authentication technique involves the use of a secret value (Figure 11.2c). The secret value itself is not sent. However, if the

Table 11.1 Requirements for a Cryptographic Hash Function H

Requirement	Description
Variable input size	H can be applied to a block of data of any size.
Fixed output size	H produces a fixed-length output.
Efficiency	$H(x)$ is relatively easy to compute for any given x , making both hardware and software implementations practical.
Preimage resistant (one-way property)	For any given hash value h , it is computationally infeasible to find y such that $H(y) = h$.
Second preimage resistant (weak collision resistant)	For any given block x , it is computationally infeasible to find $y \neq x$ with $H(y) = H(x)$.
Collision resistant (strong collision resistant)	It is computationally infeasible to find any pair (x, y) such that $H(x) = H(y)$.
Pseudorandomness	Output of H meets standard tests for pseudorandomness.

hash function is not one way, an attacker can easily discover the secret value: If the attacker can observe or intercept a transmission, the attacker obtains the message M , and the hash code $h = H(S \| M)$. The attacker then inverts the hash function to obtain $S \| M = H^{-1}(MD_M)$. Because the attacker now has both M and $S_{AB} \| M$, it is a trivial matter to recover S_{AB} .

The fifth property, **second preimage resistant**, guarantees that it is impossible to find an alternative message with the same hash value as a given message. This prevents forgery when an encrypted hash code is used (Figure 11.2b and Figure 11.3a). If this property were not true, an attacker would be capable of the following sequence: First, observe or intercept a message plus its encrypted hash code; second, generate an unencrypted hash code from the message; third, generate an alternate message with the same hash code.

A hash function that satisfies the first five properties in Table 11.1 is referred to as a weak hash function. If the sixth property, **collision resistant**, is also satisfied, then it is referred to as a strong hash function. A strong hash function protects against an attack in which one party generates a message for another party to sign. For example, suppose Bob writes an IOU message, sends it to Alice, and she signs it. Bob finds two messages with the same hash, one of which requires Alice to pay a small amount and one that requires a large payment. Alice signs the first message, and Bob is then able to claim that the second message is authentic.

Figure 11.5 shows the relationships among the three resistant properties. A function that is collision resistant is also second preimage resistant, but the reverse is not necessarily true. A function can be collision resistant but not preimage resistant and vice versa. A function can be collision resistant but not second preimage resistant and vice versa. See [MENE97] for a discussion.

Table 11.2 shows the resistant properties required for various hash function applications.

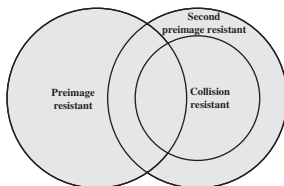


Figure 11.5 Relationship Among Hash Function Properties

The final requirement in Table 11.1, **pseudorandomness**, has not traditionally been listed as a requirement of cryptographic hash functions but is more or less implied. [JOHN05] points out that cryptographic hash functions are commonly used for key derivation and pseudorandom number generation, and that in message integrity applications, the three resistant properties depend on the output of the hash function appearing to be random. Thus, it makes sense to verify that in fact a given hash function produces pseudorandom output.

Brute-Force Attacks

As with encryption algorithms, there are two categories of attacks on hash functions: brute-force attacks and cryptanalysis. A brute-force attack does not depend on the specific algorithm but depends only on bit length. In the case of a hash function, a brute-force attack depends only on the bit length of the hash value. A cryptanalysis, in contrast, is an attack based on weaknesses in a particular cryptographic algorithm. We look first at brute-force attacks.

Table 11.2 Hash Function Resistance Properties Required for Various Data Integrity Applications

	Preimage Resistant	Second Preimage Resistant	Collision Resistant
Hash + digital signature	yes	yes	yes*
Intrusion detection and virus detection		yes	
Hash + symmetric encryption			
One-way password file	yes		
MAC	yes	yes	yes*

* Resistance required if attacker is able to mount a chosen message attack

PREIMAGE AND SECOND PREIMAGE ATTACKS For a preimage or second preimage attack, an adversary wishes to find a value y such that $H(y)$ is equal to a given hash value h . The brute-force method is to pick values of y at random and try each value until a collision occurs. For an m -bit hash value, the level of effort is proportional to 2^m . Specifically, the adversary would have to try, on average, 2^{m-1} values of y to find one that generates a given hash value h . This result is derived in Appendix 11A [Equation (11.1)].

COLLISION RESISTANT ATTACKS For a collision resistant attack, an adversary wishes to find two messages or data blocks, x and y , that yield the same hash function: $H(x) = H(y)$. This turns out to require considerably less effort than a preimage or second preimage attack. The effort required is explained by a mathematical result referred to as the **birthday paradox**. In essence, if we choose random variables from a uniform distribution in the range 0 through $N - 1$, then the probability that a repeated element is encountered exceeds 0.5 after $\sqrt{N}$ choices have been made. Thus, for an m -bit hash value, if we pick data blocks at random, we can expect to find two data blocks with the same hash value within $\sqrt{2^m} = 2^{m/2}$ attempts. The mathematical derivation of this result is found in Appendix 11A.

Yuval proposed the following strategy to exploit the birthday paradox in a collision resistant attack [YUVA79].

1. The source, A, is prepared to sign a legitimate message x by appending the appropriate m -bit hash code and encrypting that hash code with A's private key (Figure 11.3a).
2. The opponent generates $2^{m/2}$ variations x' of x , all of which convey essentially the same meaning, and stores the messages and their hash values.
3. The opponent prepares a fraudulent message y for which A's signature is desired.
4. The opponent generates minor variations y' of y , all of which convey essentially the same meaning. For each y' , the opponent computes $H(y')$, checks for matches with any of the $H(x')$ values, and continues until a match is found. That is, the process continues until a y' is generated with a hash value equal to the hash value of one of the x' values.
5. The opponent offers the valid variation to A for signature. This signature can then be attached to the fraudulent variation for transmission to the intended recipient. Because the two variations have the same hash code, they will produce the same signature; the opponent is assured of success even though the encryption key is not known.

Thus, if a 64-bit hash code is used, the level of effort required is only on the order of 2^{32} [see Appendix 11A, Equation (11.7)].

The generation of many variations that convey the same meaning is not difficult. For example, the opponent could insert a number of "space-space-backspace" character pairs between words throughout the document. Variations could then be generated by substituting "space-backspace-space" in selected instances. Alternatively, the

Dear Anthony,

{ This letter is } to introduce { you to } { Mr. } Alfred { P. }
 { I am writing } { to you } { -- }
 Barton, the { new } { chief } jewellery buyer for { our }
 { newly appointed } { senior } { the }
 Northern { European } { area } . He { will take } over { the }
 { Europe } { division } { has taken } { -- }
 responsibility for { the all } our interests in { watches and jewellery }
 { the whole of } { jewellery and watches }
 in the { area } . Please { afford } him { every } help he { may need }
 { region } { give } { all the } { needs }
 to { seek out } the most { modern } lines for the { top } end of the
 { find } { up to date } { high }
 market. He is { empowered } to receive on our behalf { samples } of the
 { authorized } { specimens }
 { latest } { watch and jewellery } products, { up } to a { limit }
 { newest } { jewellery and watch } { subject } { maximum }
 of ten thousand dollars. He will { carry } a signed copy of this { letter }
 { hold } { document }
 as proof of identity. An order with his signature, which is { appended }
 { attached }
 { authorizes } you to charge the cost to this company at the { above }
 { allows } { head office }
 address. We { fully } expect that our { level } of orders will increase in
 { -- } { volume }
 the { following } year and { trust } that the new appointment will { be }
 { next } { hope } { prove }
 { advantageous } to both our companies.
 { an advantage }

Figure 11.6 A Letter in 2^{37} Variation [DAVI89]

opponent could simply reword the message but retain the meaning. Figure 11.6 [DAVI89] provides an example.

To summarize, for a hash code of length m , the level of effort required, as we have seen, is proportional to the following.

Preimage resistant	2^m
Second preimage resistant	2^m
Collision resistant	$2^{m/2}$

If collision resistance is required (and this is desirable for a general-purpose secure hash code), then the value $2^{m/2}$ determines the strength of the hash code against brute-force attacks. Van Oorschot and Wiener [VANO94] presented a design for a \$10 million collision search machine for MD5, which has a 128-bit hash length, that could find a collision in 24 days. Thus, a 128-bit code may be viewed as inadequate. The next step up, if a hash code is treated as a sequence of 32 bits, is a 160-bit hash length. With a hash length of 160 bits, the same search machine would require over four thousand years to find a collision. With today's technology, the time would be much shorter, so that 160 bits now appears suspect.

Cryptanalysis

As with encryption algorithms, cryptanalytic attacks on hash functions seek to exploit some property of the algorithm to perform some attack other than an exhaustive search. The way to measure the resistance of a hash algorithm to cryptanalysis is to compare its strength to the effort required for a brute-force attack. That is, an ideal hash algorithm will require a cryptanalytic effort greater than or equal to the brute-force effort.

In recent years, there has been considerable effort, and some successes, in developing cryptanalytic attacks on hash functions. To understand these, we need to look at the overall structure of a typical secure hash function, indicated in Figure 11.7. This structure, referred to as an iterated hash function, was proposed by Merkle [MERK79, MERK89] and is the structure of most hash functions in use today, including SHA, which is discussed later in this chapter. The hash function takes an input message and partitions it into L fixed-sized blocks of b bits each. If necessary, the final block is padded to b bits. The final block also includes the value of the total length of the input to the hash function. The inclusion of the length makes the job of the opponent more difficult. Either the opponent must find two messages of equal length that hash to the same value or two messages of differing lengths that, together with their length values, hash to the same value.

The hash algorithm involves repeated use of a **compression function**, f , that takes two inputs (an n -bit input from the previous step, called the *chaining variable*, and a b -bit block) and produces an n -bit output. At the start of hashing, the chaining variable has an initial value that is specified as part of the algorithm. The final value

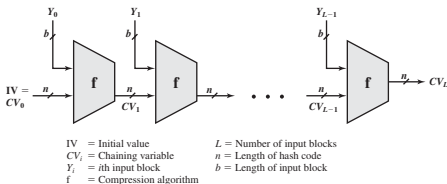


Figure 11.7 General Structure of Secure Hash Code

of the chaining variable is the hash value. Often, $b > n$; hence the term *compression*. The hash function can be summarized as

$$\begin{aligned} CV_0 &= IV = \text{initial } n\text{-bit value} \\ CV_i &= f(CV_{i-1}, Y_{i-1}) \quad 1 \leq i \leq L \\ H(M) &= CV_L \end{aligned}$$

where the input to the hash function is a message M consisting of the blocks $Y_0, Y_1, \dots, Y_{L-1}$.

The motivation for this iterative structure stems from the observation by Merkle [MERK89] and Damgård [DAMG89] that if the compression function is collision resistant, then so is the resultant iterated hash function.¹ Therefore, the structure can be used to produce a secure hash function to operate on a message of any length. The problem of designing a secure hash function reduces to that of designing a collision-resistant compression function that operates on inputs of some fixed size.

Cryptanalysis of hash functions focuses on the internal structure of f and is based on attempts to find efficient techniques for producing collisions for a single execution of f . Once that is done, the attack must take into account the fixed value of IV . The attack on f depends on exploiting its internal structure. Typically, as with symmetric block ciphers, f consists of a series of rounds of processing, so that the attack involves analysis of the pattern of bit changes from round to round.

Keep in mind that for any hash function there must exist collisions, because we are mapping a message of length at least equal to twice the block size b (because we must append a length field) into a hash code of length n , where $b \geq n$. What is required is that it is computationally infeasible to find collisions.

The attacks that have been mounted on hash functions are rather complex and beyond our scope here. For the interested reader, [DOBB96] and [BELL97] are recommended.

11.4 HASH FUNCTIONS BASED ON CIPHER BLOCK CHAINING

A number of proposals have been made for hash functions based on using a cipher block chaining technique, but without using the secret key. One of the first such proposals was that of Rabin [RAB178]. Divide a message M into fixed-size blocks $M_1, M_2, \dots, M_N$ and use a symmetric encryption system such as DES to compute the hash code G as

$$\begin{aligned} H_0 &= \text{initial value} \\ H_i &= E(M_i, H_{i-1}) \\ G &= H_N \end{aligned}$$

This is similar to the CBC technique, but in this case, there is no secret key. As with any hash code, this scheme is subject to the birthday attack, and if the encryption algorithm is DES and only a 64-bit hash code is produced, then the system is vulnerable.

¹The converse is not necessarily true.

Furthermore, another version of the birthday attack can be used even if the opponent has access to only one message and its valid signature and cannot obtain multiple signings. Here is the scenario: We assume that the opponent intercepts a message with a signature in the form of an encrypted hash code and that the unencrypted hash code is m bits long.

1. Use the algorithm defined at the beginning of this subsection to calculate the unencrypted hash code G .
2. Construct any desired message in the form $Q_1, Q_2, \dots, Q_{N-2}$.
3. Compute $H_i = E(Q_i, H_{i-1})$ for $1 \leq i \leq (N-2)$.
4. Generate $2^{m/2}$ random blocks; for each block X , compute $E(X, H_{N-2})$. Generate an additional $2^{m/2}$ random blocks; for each block Y , compute $D(Y, G)$, where D is the decryption function corresponding to E .
5. Based on the birthday paradox, with high probability there will be an X and Y such that $E(X, H_{N-2}) = D(Y, G)$.
6. Form the message $Q_1, Q_2, \dots, Q_{N-2}, X, Y$. This message has the hash code G and therefore can be used with the intercepted encrypted signature.

This form of attack is known as a **meet-in-the-middle-attack**. A number of researchers have proposed refinements intended to strengthen the basic block chaining approach. For example, Davies and Price [DAVI89] describe the variation:

$$H_i = E(M_i, H_{i-1}) \oplus H_{i-1}$$

Another variation, proposed in [MEYE88], is

$$H_i = E(H_{i-1}, M_i) \oplus M_i$$

However, both of these schemes have been shown to be vulnerable to a variety of attacks [MIYA90]. More generally, it can be shown that some form of birthday attack will succeed against any hash scheme involving the use of cipher block chaining without a secret key, provided that either the resulting hash code is small enough (e.g., 64 bits or less) or that a larger hash code can be decomposed into independent subcodes [JUEN87].

Thus, attention has been directed at finding other approaches to hashing. Many of these have also been shown to have weaknesses [MITC92].

11.5 SECURE HASH ALGORITHM (SHA)

In recent years, the most widely used hash function has been the Secure Hash Algorithm (SHA). Indeed, because virtually every other widely used hash function had been found to have substantial cryptanalytic weaknesses, SHA was more or less the last remaining standardized hash algorithm by 2005. SHA was developed by the National Institute of Standards and Technology (NIST) and published

as a federal information processing standard (FIPS 180) in 1993. When weaknesses were discovered in SHA, now known as **SHA-0**, a revised version was issued as FIPS 180-1 in 1995 and is referred to as **SHA-1**. The actual standards document is entitled “Secure Hash Standard.” SHA is based on the hash function MD4, and its design closely models MD4. SHA-1 is also specified in RFC 3174, which essentially duplicates the material in FIPS 180-1 but adds a C code implementation.

SHA-1 produces a hash value of 160 bits. In 2002, NIST produced a revised version of the standard, FIPS 180-2, that defined three new versions of SHA, with hash value lengths of 256, 384, and 512 bits, known as SHA-256, SHA-384, and SHA-512, respectively. Collectively, these hash algorithms are known as **SHA-2**. These new versions have the same underlying structure and use the same types of modular arithmetic and logical binary operations as SHA-1. A revised document was issued as FIP PUB 180-3 in 2008, which added a 224-bit version (Table 11.3). SHA-2 is also specified in RFC 4634, which essentially duplicates the material in FIPS 180-3 but adds a C code implementation.

In 2005, NIST announced the intention to phase out approval of SHA-1 and move to a reliance on SHA-2 by 2010. Shortly thereafter, a research team described an attack in which two separate messages could be found that deliver the same SHA-1 hash using 2^{69} operations, far fewer than the 2^{80} operations previously thought needed to find a collision with an SHA-1 hash [WANG05]. This result should hasten the transition to SHA-2.

In this section, we provide a description of SHA-512. The other versions are quite similar.

SHA-512 Logic

The algorithm takes as input a message with a maximum length of less than 2^{128} bits and produces as output a 512-bit message digest. The input is processed in 1024-bit blocks. Figure 11.8 depicts the overall processing of a message to produce a digest. This follows the general structure depicted in Figure 11.7. The processing consists of the following steps.

Table 11.3 Comparison of SHA Parameters

	SHA-1	SHA-224	SHA-256	SHA-384	SHA-512
Message Digest Size	160	224	256	384	512
Message Size	$< 2^{64}$	$< 2^{64}$	$< 2^{64}$	$< 2^{128}$	$< 2^{128}$
Block Size	512	512	512	1024	1024
Word Size	32	32	32	64	64
Number of Steps	80	64	64	80	80

Note: All sizes are measured in bits.

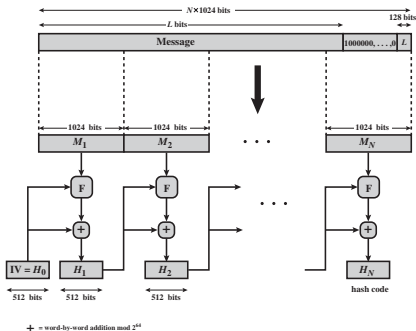


Figure 11.8 Message Digest Generation Using SHA-512

Step 1 Append padding bits. The message is padded so that its length is congruent to 896 modulo 1024 [$\text{length} \equiv 896 \pmod{1024}$]. Padding is always added, even if the message is already of the desired length. Thus, the number of padding bits is in the range of 1 to 1024. The padding consists of a single 1 bit followed by the necessary number of 0 bits.

Step 2 Append length. A block of 128 bits is appended to the message. This block is treated as an unsigned 128-bit integer (most significant byte first) and contains the length of the original message (before the padding).

The outcome of the first two steps yields a message that is an integer multiple of 1024 bits in length. In Figure 11.8, the expanded message is represented as the sequence of 1024-bit blocks $M_1, M_2, \dots, M_N$, so that the total length of the expanded message is $N \times 1024$ bits.

Step 3 Initialize hash buffer. A 512-bit buffer is used to hold intermediate and final results of the hash function. The buffer can be represented as eight 64-bit registers (a, b, c, d, e, f, g, h). These registers are initialized to the following 64-bit integers (hexadecimal values):

```

a = 6A09E667F3BCC908   e = 510E527FADE682D1
b = BB67AE8584CAA73B   f = 9B05688C2B3E6C1F
c = 3C6EF372FE94F82B   g = 1F83D9ABFB41BD6B
d = A54FF53A5F1D36F1   h = 5BE0CD19137E2179

```

These values are stored in **big-endian** format, which is the most significant byte of a word in the low-address (leftmost) byte position. These words were obtained by taking the first sixty-four bits of the fractional parts of the square roots of the first eight prime numbers.

Step 4 Process message in 1024-bit (128-word) blocks. The heart of the algorithm is a module that consists of 80 rounds; this module is labeled F in Figure 11.8. The logic is illustrated in Figure 11.9.

Each round takes as input the 512-bit buffer value, $abcdefgh$, and updates the contents of the buffer. At input to the first round, the buffer has the value of the intermediate hash value, H_{i-1} . Each round t makes use of a 64-bit value W_t , derived from the current 1024-bit block being processed (M_i). These values are derived using a message schedule described subsequently. Each round also makes use of an additive constant K_t , where $0 \leq t \leq 79$ indicates one of the 80 rounds. These words represent the first 64 bits of the fractional parts of the cube roots of the first 80 prime numbers. The constants provide a “randomized” set of 64-bit patterns, which should eliminate any regularities in the input data. Table 11.4 shows these constants in hexadecimal format (from left to right).

The output of the eightieth round is added to the input to the first round (H_{i-1}) to produce H_i . The addition is done independently for each of the eight

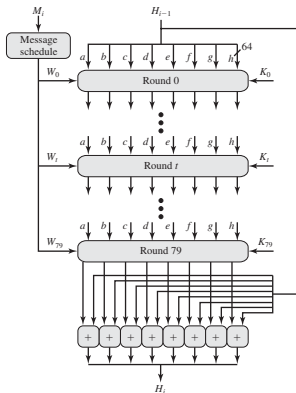


Figure 11.9 SHA-512 Processing of a Single 1024-Bit Block

words in the buffer with each of the corresponding words in H_{i-1} , using addition modulo 2^{64} .

Step 5 Output. After all N 1024-bit blocks have been processed, the output from the N th stage is the 512-bit message digest.

We can summarize the behavior of SHA-512 as follows:

$$H_0 = \text{IV}$$

$$H_i = \text{SUM}_{64}(H_{i-1}, \text{abcdefgh}_i)$$

$$MD = H_N$$

where

IV = initial value of the abcdefgh buffer, defined in step 3

abcdefgh_{*i*} = the output of the last round of processing of the *i*th message block

N = the number of blocks in the message (including padding and length fields)

SUM₆₄ = addition modulo 2^{64} performed separately on each word of the pair of inputs

MD = final message digest value

SHA-512 Round Function

Let us look in more detail at the logic in each of the 80 steps of the processing of one 512-bit block (Figure 11.10). Each round is defined by the following set of equations:

$$T_1 = h + \text{Ch}(e, f, g) + \left(\sum_{i=1}^{512} e\right) + W_t + K_t$$

$$T_2 = \left(\sum_{i=0}^{512} a\right) + \text{Maj}(a, b, c)$$

$$h = g$$

$$g = f$$

$$f = e$$

$$e = d + T_1$$

$$d = c$$

$$c = b$$

$$b = a$$

$$a = T_1 + T_2$$

where

t = step number; $0 \leq t \leq 79$

$\text{Ch}(e, f, g) = (e \text{ AND } f) \oplus (\text{NOT } e \text{ AND } g)$
the conditional function: If *e* then *f* else *g*

Table 11.4 SHA-512 Constants

428a2f98d728ae22	7137449123ef65cd	b5c0fbfec4d3b2f	e9b5dba58189dbbc
3956c25bf348b538	59f111f1b605d019	923f82a4af194f9b	ab1c5ed5da6d8118
d807aa98a3030242	12835b0145706fbc	243185be4ee4b28c	550c7dc3d5ffb4e2
72be5d74f27b896f	80deb1fe3b1696b1	9bdc06a725c71235	c19bf174cf692694
e49b69c19ef14ad2	efbe4786384f25e3	0fc19dc68b8cd5b5	240ca1cc77ac9c65
2de92c6f592b0275	4a7484aa6ea6e483	5cb0a9dcbd41fbd4	76f988da831153b5
983e5152ee66dfab	a831c66d2db43210	b00327c898fb213f	bf597fc7beef0ee4
c6e00bf33da88fc2	d5a79147930aa725	06ca6351e003826f	142929670a0e6e70
27b70a8546d22ffc	2e1b21385c26c926	4d2c6dfc5ac42aed	53380d13995b3df
650a73548baf63de	766a0abb3c77b2a8	81c2c92e47edaee6	92722c851482353b
a2bfe8a14cf10364	a81a664bbc423001	c24b8b70d0f89791	c76c51a30654be30
d192e819d6ef5218	d69906245565a910	f40e3585771202a	106aa07032bbd1b8
19a4c116b8d2d0c8	1e376c085141ab53	2748774cdf8eeb99	34b0cbb5e19b48a8
391c0cb3c5c95a63	4ed8aa4ae3418acb	5b9cca4f7763e373	682e6ff3d6b2b8a3
748f82ee5defb2fc	78a5636f43172f60	84c87814a1f0ab72	8cc702081a6439ec
90befffa23631e28	a4506cebd82bde9	bef9a3f7b2c67915	c67178f2e372532b
ca273ecee26619c	d186b8c721c0c207	eadad7dd6cde0eb1e	f57d4f7fee6ed178
06f067aa72176fba	0a637dc5a2c898a6	113f9804bef90dae	1b710b35131c471b
28db77f523047d84	32caab7b40c72493	3c9ebe0a15c9bebc	431d67c49c100d4c
4cc5d4becb3e42b6	597f299cfc657e2a	5fcb6fab3ad6faec	6c44198c4a475817

$\text{Maj}(a, b, c) = (a \text{ AND } b) \oplus (a \text{ AND } c) \oplus (b \text{ AND } c)$
the function is true only if the majority (two or three) of the arguments are true

$$\left(\sum_0^{512} a\right) = \text{ROTR}^{28}(a) \oplus \text{ROTR}^{34}(a) \oplus \text{ROTR}^{39}(a)$$

$$\left(\sum_1^{512} e\right) = \text{ROTR}^{14}(e) \oplus \text{ROTR}^{18}(e) \oplus \text{ROTR}^{41}(e)$$

$\text{ROTR}^n(x)$ = circular right shift (rotation) of the 64-bit argument x by n bits

W_t = a 64-bit word derived from the current 512-bit input block

K_t = a 64-bit additive constant

$+$ = addition modulo 2^{64}

Two observations can be made about the round function.

1. Six of the eight words of the output of the round function involve simply permutation (b, c, d, f, g, h) by means of rotation. This is indicated by shading in Figure 11.10.
2. Only two of the output words (a, e) are generated by substitution. Word e is a function of input variables (d, e, f, g, h), as well as the round word W_t and the constant K_t . Word a is a function of all of the input variables except d , as well as the round word W_t and the constant K_t .

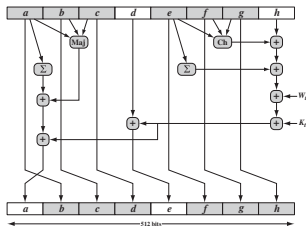


Figure 11.10 Elementary SHA-512 Operation (single round)

It remains to indicate how the 64-bit word values W_t are derived from the 1024-bit message. Figure 11.11 illustrates the mapping. The first 16 values of W_t are taken directly from the 16 words of the current block. The remaining values are defined as

$$W_t = \sigma_1^{512}(W_{t-2}) + W_{t-7} + \sigma_0^{512}(W_{t-15}) + W_{t-16}$$

where

$$\sigma_0^{512}(x) = \text{ROTR}^1(x) \oplus \text{ROTR}^8(x) \oplus \text{SHR}^7(x)$$

$$\sigma_1^{512}(x) = \text{ROTR}^{19}(x) \oplus \text{ROTR}^{61}(x) \oplus \text{SHR}^6(x)$$

$\text{ROTR}^n(x)$ = circular right shift (rotation) of the 64-bit argument x by n bits

$\text{SHR}^n(x)$ = left shift of the 64-bit argument x by n bits with padding by zeros on the right

$+$ = addition modulo 2^{64}

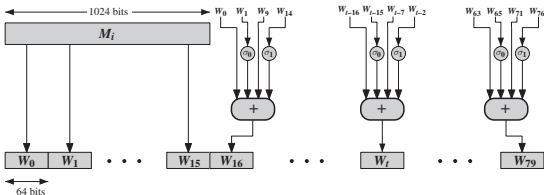


Figure 11.11 Creation of 80-word Input Sequence for SHA-512 Processing of Single Block

Thus, in the first 16 steps of processing, the value of W_i is equal to the corresponding word in the message block. For the remaining 64 steps, the value of W_i consists of the circular left shift by one bit of the XOR of four of the preceding values of W_i , with two of those values subjected to shift and rotate operations. This introduces a great deal of redundancy and interdependence into the message blocks that are compressed, which complicates the task of finding a different message block that maps to the same compression function output.

Figure 11.12 summarizes the SHA-512 logic.

The SHA-512 algorithm has the property that every bit of the hash code is a function of every bit of the input. The complex repetition of the basic function F produces results that are well mixed; that is, it is unlikely that two messages chosen at random, even if they exhibit similar regularities, will have the same hash code. Unless there is some hidden weakness in SHA-512, which has not so far been published, the difficulty of coming up with two messages having the same message digest is on the order of 2^{256} operations, while the difficulty of finding a message with a given digest is on the order of 2^{512} operations.

Example

We include here an example based on one in FIPS 180. We wish to hash a one-block message consisting of three ASCII characters: “abc”, which is equivalent to the following 24-bit binary string:

01100001 01100010 01100011

Recall from step 1 of the SHA algorithm, that the message is padded to a length congruent to 896 modulo 1024. In this case of a single block, the padding consists of $896 - 24 = 872$ bits, consisting of a “1” bit followed by 871 “0” bits. Then a 128-bit length value is appended to the message, which contains the length of the original message (before the padding). The original length is 24 bits, or a hexadecimal value of 18. Putting this all together, the 1024-bit message block, in hexadecimal, is

```
6162638000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000018
```

This block is assigned to the words $W_0, \dots, W_{15}$ of the message schedule, which appears as follows.

$W_0 = 6162638000000000$	$W_5 = 0000000000000000$
$W_1 = 0000000000000000$	$W_6 = 0000000000000000$
$W_2 = 0000000000000000$	$W_7 = 0000000000000000$
$W_3 = 0000000000000000$	$W_8 = 0000000000000000$
$W_4 = 0000000000000000$	$W_9 = 0000000000000000$

$$W_{10} = 0000000000000000$$

$$W_{13} = 0000000000000000$$

$$W_{11} = 0000000000000000$$

$$W_{14} = 0000000000000000$$

$$W_{12} = 0000000000000000$$

$$W_{15} = 0000000000000018$$

As indicated in Figure 11.12, the eight 64-bit variables, a through h , are initialized to values $H_{0,0}$ through $H_{0,7}$. The following table shows the initial values of these variables and their values after each of the first two rounds.

a	6a09e667f3bcc908	f6afceb8bcfcddf5	1320f8c9fb872cc0
b	bb67ae8584caa73b	6a09e667f3bcc908	f6afceb8bcfcddf5
c	3c6ef372fe94f82b	bb67ae8584caa73b	6a09e667f3bcc908
d	a54ff53a5f1d36f1	3c6ef372fe94f82b	bb67ae8584caa73b
e	510e527fade682d1	58cb02347ab51f91	c3d4ebfd48650ffa
f	9b05688c2b3e6c1f	510e527fade682d1	58cb02347ab51f91
g	1f83d9abfb41bd6b	9b05688c2b3e6c1f	510e527fade682d1
h	5be0cd19137e2179	1f83d9abfb41bd6b	9b05688c2b3e6c1f

Note that in each of the rounds, six of the variables are copied directly from variables from the preceding round.

The process continues through 80 rounds. The output of the final round is

```
73a54f399fa4b1b2 10d9c4c4295599f6 d67806db8b148677 654ef9abec389ca9
d08446aa79693ed7 9bb4d39778c07f9e 25c96a7768fb2aa3 ceb9fc3691ce8326
```

The hash value is then calculated as

$$H_{1,0} = 6a09e667f3bcc908 + 73a54f399fa4b1b2 = ddaf35a193617aba$$

$$H_{1,1} = bb67ae8584caa73b + 10d9c4c4295599f6 = cc417349ae204131$$

$$H_{1,2} = 3c6ef372fe94f82b + d67806db8b148677 = 12e6fa4e89a97ea2$$

$$H_{1,3} = a54ff53a5f1d36f1 + 654ef9abec389ca9 = 0a9eeee64b55d39a$$

$$H_{1,4} = 510e527fade682d1 + d08446aa79693ed7 = 2192992a274fcl8$$

$$H_{1,5} = 9b05688c2b3e6c1f + 9bb4d39778c07f9e = 36ba3c23a3feebbd$$

$$H_{1,6} = 1f83d9abfb41bd6b + 25c96a7768fb2aa3 = 454d4423643ce80e$$

$$H_{1,7} = 5be0cd19137e2179 + ceb9fc3691ce8326 = 2a9ac94fa54ca49f$$

The resulting 512-bit message digest is

```
ddaf35a193617aba cc417349ae204131 12e6fa4e89a97ea2 0a9eeee64b55d39a
2192992a274fcl8 36ba3c23a3feebbd 454d4423643ce80e 2a9ac94fa54ca49f
```


The padded message consists blocks $M_1, M_2, \dots, M_N$. Each message block M_i consists of 16 64-bit words $M_{i,0}, M_{i,1}, \dots, M_{i,15}$. All addition is performed modulo 2^{64} .

$$\begin{array}{ll} H_{0,0} = 6A09E667F3BCC908 & H_{0,4} = 510E527FADE682D1 \\ H_{0,1} = BB67AE8584CAA73B & H_{0,5} = 9B05688C2B3E6C1F \\ H_{0,2} = 3C6EF372FE94F82B & H_{0,6} = 1F83D9ABFB41BD6B \\ H_{0,3} = A54FF53A5F1D36F1 & H_{0,7} = 5BE0CDI9137E2179 \end{array}$$

for $i = 1$ **to** N

1. Prepare the message schedule W

for $t = 0$ **to** 15

$$W_t = M_{i,t}$$

for $t = 16$ **to** 79

$$W_t = \sigma_1^{512}(W_{t-2}) + W_{t-7} + \sigma_0^{512}(W_{t-15}) + W_{t-16}$$

2. Initialize the working variables

$$a = H_{i-1,0} \quad e = H_{i-1,4}$$

$$b = H_{i-1,1} \quad f = H_{i-1,5}$$

$$c = H_{i-1,2} \quad g = H_{i-1,6}$$

$$d = H_{i-1,3} \quad h = H_{i-1,7}$$

3. Perform the main hash computation

for $t = 0$ **to** 79

$$T_1 = h + \text{Ch}(e, f, g) + \left(\sum_1^{512} e \right) + W_t + K_t$$

$$T_2 = \left(\sum_0^{512} a \right) + \text{Maj}(a, b, c)$$

$$h = g$$

$$g = f$$

$$f = e$$

$$e = d + T_1$$

$$d = c$$

$$c = b$$

$$b = a$$

$$a = T_1 + T_2$$

4. Compute the intermediate hash value

$$H_{i,0} = a + H_{i-1,0} \quad H_{i,4} = a + H_{i-1,4}$$

$$H_{i,1} = a + H_{i-1,1} \quad H_{i,5} = a + H_{i-1,5}$$

$$H_{i,2} = a + H_{i-1,2} \quad H_{i,6} = a + H_{i-1,6}$$

$$H_{i,3} = a + H_{i-1,3} \quad H_{i,7} = a + H_{i-1,7}$$

return $\{H_{N,0} \parallel H_{N,1} \parallel H_{N,2} \parallel H_{N,3} \parallel H_{N,4} \parallel H_{N,5} \parallel H_{N,6} \parallel H_{N,7}\}$

Figure 11.12 SHA-512 Logic

Suppose now that we change the input message by one bit, from “abc” to “cbc”. Then, the 1024-bit message block is

```
6362638000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000018
```

And the resulting 512-bit message digest is

```
531668966ee79b70 0b8e593261101354 4273f7ef7b31f279 2a7ef68d53f93264
319c165ad96d9187 55e6a204c2607e27 6e05cdf993a64c85 ef9e1e125c0f925f
```

The number of bit positions that differ between the two hash values is 253, almost exactly half the bit positions, indicating that SHA-512 has a good avalanche effect.

11.6 SHA-3

As of this writing, SHA-1 has not yet been “broken.” That is, no one has demonstrated a technique for producing collisions in less than brute-force time. However, because SHA-1 is very similar in structure and in the basic mathematical operations used to MD5 and SHA-0, both of which have been broken, SHA-1 is considered insecure and has been phased out for SHA-2.

SHA-2, particularly the 512-bit version, would appear to provide unassailable security. However, SHA-2 shares the same structure and mathematical operations as its predecessors, and this is a cause for concern. Because it will take years to find a suitable replacement for SHA-2, should it become vulnerable, NIST decided to begin the process of developing a new hash standard.

Accordingly, NIST announced in 2007 a competition to produce the next generation NIST hash function, to be called SHA-3. NIST would like to have a new standard in place by the end of 2012, but emphasizes that this is not a fixed timeline and that the schedule could slip well beyond that date. The basic requirements that must be satisfied by any candidate for SHA-3 are the following.

1. It must be possible to replace SHA-2 with SHA-3 in any application by a simple drop-in substitution. Therefore, SHA-3 must support hash value lengths of 224, 256, 384, and 512 bits.
2. SHA-3 must preserve the online nature of SHA-2. That is, the algorithm must process comparatively small blocks (512 or 1024 bits) at a time instead of requiring that the entire message be buffered in memory before processing it.

Beyond these basic requirements, NIST has defined a set of evaluation criteria. These criteria are designed to reflect the requirements for the main applications supported by SHA-2, which include digital signatures, hashed message authentication

codes, key generation, and pseudorandom number generation. The evaluation criteria for the new hash function, in decreasing order of importance, are as follows.

- **Security:** The security strength of SHA-3 should be close to the theoretical maximum for the different required hash sizes and for both preimage resistance and collision resistance. SHA-3 algorithms must be designed to resist any potentially successful attack on SHA-2 functions. In practice, this probably means that SHA-3 must be fundamentally different than the SHA-1, SHA-2, and MD5 algorithms in either structure, mathematical functions, or both.
- **Cost:** SHA-3 should be both time and memory efficient over a range of hardware platforms.
- **Algorithm and implementation characteristics:** Consideration will be given to such characteristics as flexibility (e.g., tunable parameters for security/performance tradeoffs, opportunity for parallelization, and so on) and simplicity. The latter characteristic makes it easier to analyze the security properties of the algorithm

11.7 RECOMMENDED READING AND WEB SITES

[PREN99] is a good survey of cryptographic hash functions. [GILB03] examines the security of SHA-256 through SHA-512.

GILB03 Gilbert, H. and Handschuh, H. "Security Analysis of SHA-256 and Sisters." *Proceedings, CRYPTO '03*, 2003; published by Springer-Verlag.

PREN99 Preneel, B. "The State of Cryptographic Hash Functions." *Proceedings, EURO-CRYPT '96*, 1996; published by Springer-Verlag.



Recommended Web Sites:

- **NIST Secure Hashing Page:** SHA FIPS and related documents.
- **Cryptographic Hash Algorithm Competition:** NIST page on its competition for a new standardized hash algorithm, to be called SHA-3.

11.8 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

Key Terms

big endian birthday attack	birthday paradox collision resistant	compression function cryptographic hash function
-------------------------------	---	---

hash code	MD4	SHA-224
hash function	MD5	SHA-256
hash value	message digest	SHA-384
keyed hash function	one-way hash function	SHA-512
little endian	preimage resistant	strong collision resistance
message authentication code (MAC)	second preimage resistant	weak collision resistance
	SHA-1	

Review Questions

- 11.1 What characteristics are needed in a secure hash function?
- 11.2 What is the difference between weak and strong collision resistance?
- 11.3 What is the role of a compression function in a hash function?
- 11.4 What is the difference between little-endian and big-endian format?
- 11.5 What basic arithmetical and logical functions are used in SHA?

Problems

- 11.1 The high-speed transport protocol XTP (Xpress Transfer Protocol) uses a 32-bit checksum function defined as the concatenation of two 16-bit functions: XOR and RXOR, defined in Section 11.4 as “two simple hash functions” and illustrated in Figure 11.4.

- a. Will this checksum detect all errors caused by an odd number of error bits? Explain.
- b. Will this checksum detect all errors caused by an even number of error bits? If not, characterize the error patterns that will cause the checksum to fail.
- c. Comment on the effectiveness of this function for use as a hash function for authentication.

- 11.2 a. Consider the Davies and Price hash code scheme described in Section 11.4 and assume that DES is used as the encryption algorithm:

$$H_i = H_{i-1} \oplus E(M_i, H_{i-1})$$

Recall the complementarity property of DES (Problem 3.14): If $Y = E(K, X)$, then $Y' = E(K', X')$. Use this property to show how a message consisting of blocks $M_1, M_2, \dots, M_N$ can be altered without altering its hash code.

- b. Show that a similar attack will succeed against the scheme proposed in [MEYE88]:

$$H_i = M_i \oplus E(H_{i-1}, M_i)$$

- 11.3 a. Consider the following hash function. Messages are in the form of a sequence of numbers in $\mathbb{Z}_n$, $M = (a_1, a_2, \dots, a_i)$. The hash value h is calculated as $\left(\sum_{i=1}^i a_i\right)$ for some predefined value n . Does this hash function satisfy any of the requirements for a hash function listed in Table 11.1? Explain your answer.
- b. Repeat part (a) for the hash function $h = \left(\sum_{i=1}^i (a_i)^2\right) \bmod n$.
- c. Calculate the hash function of part (b) for $M = (189, 632, 900, 722, 349)$ and $n = 989$.
- 11.4 It is possible to use a hash function to construct a block cipher with a structure similar to DES. Because a hash function is one way and a block cipher must be reversible (to decrypt), how is it possible?

- 11.5 Now consider the opposite problem: using an encryption algorithm to construct a one-way hash function. Consider using RSA with a known key. Then process a message consisting of a sequence of blocks as follows: Encrypt the first block, XOR the result with the second block and encrypt again, etc. Show that this scheme is not secure by solving the following problem. Given a two-block message B_1, B_2 , and its hash

$$\text{RSAH}(B_1, B_2) = \text{RSA}(\text{RSA}(B_1) \oplus B_2)$$

Given an arbitrary block C_1 , choose C_2 so that $\text{RSAH}(C_1, C_2) = \text{RSAH}(B_1, B_2)$. Thus, the hash function does not satisfy weak collision resistance.

- 11.6 Suppose $H(m)$ is a collision-resistant hash function that maps a message of arbitrary bit length into an n -bit hash value. Is it true that, for all messages x, x' with $x \neq x'$, we have $H(x) \neq H(x')$? Explain your answer.
- 11.7 In Figure 11.11, it is assumed that an array of 80 64-bit words is available to store the values of W_i , so that they can be precomputed at the beginning of the processing of a block. Now assume that space is at a premium. As an alternative, consider the use of a 16-word circular buffer that is initially loaded with W_0 through W_{15} . Design an algorithm that, for each step i , computes the required input value W_i .
- 11.8 For SHA-512, show the equations for the values of W_{16}, W_{17}, W_{18} , and W_{19} .
- 11.9 State the value of the padding field in SHA-512 if the length of the message is
- 1919 bits
 - 1920 bits
 - 1921 bits
- 11.10 State the value of the length field in SHA-512 if the length of the message is
- 1919 bits
 - 1920 bits
 - 1921 bits
- 11.11 Suppose $a_1 a_2 a_3 a_4$ are the 4 bytes in a 32-bit word. Each a_i can be viewed as an integer in the range 0 to 255, represented in binary. In a big-endian architecture, this word represents the integer

$$a_1 2^{24} + a_2 2^{16} + a_3 2^8 + a_4$$

In a little-endian architecture, this word represents the integer

$$a_4 2^{24} + a_3 2^{16} + a_2 2^8 + a_1$$

- Some hash functions, such as MD5, assume a little-endian architecture. It is important that the message digest be independent of the underlying architecture. Therefore, to perform the modulo 2 addition operation of MD5 or RIPEMD-160 on a big-endian architecture, an adjustment must be made. Suppose $X = x_1 x_2 x_3 x_4$ and $Y = y_1 y_2 y_3 y_4$. Show how the MD5 addition operation $(X + Y)$ would be carried out on a big-endian machine.
 - SHA assumes a big-endian architecture. Show how the operation $(X + Y)$ for SHA would be carried out on a little-endian machine.
- 11.12 This problem introduces a hash function similar in spirit to SHA that operates on letters instead of binary data. It is called the *toy tetragraph hash* (tth).² Given a message consisting of a sequence of letters, tth produces a hash value consisting of four letters. First, tth divides the message into blocks of 16 letters, ignoring spaces, punctuation, and capitalization. If the message length is not divisible by 16, it is padded out with nulls. A four-number running total is maintained that starts out with the value $(0, 0, 0, 0)$; this is input to the compression function for processing the first block. The compression function consists of two rounds.

Round 1 Get the next block of text and arrange it as a row-wise 4×4 block of text and convert it to numbers ($A = 0, B = 1$, etc.). For example, for the block ABCDEFGHIJKLMNOP, we have

²I thank William K. Mason, of the magazine staff of *The Cryptogram*, for providing this example.

A	B	C	D
E	F	G	H
I	J	K	L
M	N	O	P

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

Then, add each column mod 26 and add the result to the running total, mod 26. In this example, the running total is (24, 2, 6, 10).

Round 2 Using the matrix from round 1, rotate the first row left by 1, second row left by 2, third row left by 3, and reverse the order of the fourth row.

In our example:

B	C	D	A
G	H	E	F
L	I	J	K
P	O	N	M

1	2	3	0
6	7	4	5
11	8	9	10
15	14	13	12

Now, add each column mod 26 and add the result to the running total. The new running total is (5, 7, 9, 11). This running total is now the input into the first round of the compression function for the next block of text. After the final block is processed, convert the final running total to letters. For example, if the message is ABCDEFGHIJKLMNOP, then the hash is FHJL.

- Draw figures comparable to Figures 11.8 and 11.9 to depict the overall tth logic and the compression function logic.
- Calculate the hash function for the 48-letter message “I leave twenty million dollars to my friendly cousin Bill.”
- To demonstrate the weakness of tth, find a 48-letter block that produces the same hash as that just derived. *Hint:* Use lots of A's.

The remaining problems deal with the hash function Whirlpool, described in Appendix N.

- Develop a table similar to Table 4.9 for $GF(2^8)$ with $m(x) = x^8 + x^4 + x^3 + x^2 + 1$.
- Show the E and E^{-1} mini-boxes in Table N.2 in the traditional S-box square matrix format, such as that of Table 5.4.
- Verify that Figure N.5 is a valid implementation of the S-box shown in Table N.2a. Do this by showing the calculations involved for three input values: 00, 55, 1E.
- Provide a Boolean expression that defines the S-box functionality of Figure N.5.
- Whirlpool makes use of the construction $H_i = E(H_{i-1}, M_i) \oplus H_{i-1} \oplus M_i$. Another construction that was shown by Preneel to be secure is $H_i = E(H_{i-1}, M_i) \oplus M_i$. Now notice that the key schedule for Whirlpool resembles encryption of the cipher key under a pseudo-key defined by the round constants, so that the core of the hashing process could be formally viewed as two interacting encryption lines. Consider the encryption $E(H_{i-1}, M_i)$. We could write the final round key for this block as $K_{10} = E(RC, H_{i-1})$. Now show that the two hash constructions are essentially equivalent because of the way that the key schedule is defined.

APPENDIX 11A MATHEMATICAL BASIS OF THE BIRTHDAY ATTACK

In this appendix, we derive the mathematical justification for the birthday attack. We begin with a related problem and then look at the problem from which the name “birthday attack” is derived.

Related Problem

A general problem relating to hash functions is the following. Given a hash function H , with n possible outputs and a specific value $H(x)$, if H is applied to k random inputs, what must be the value of k so that the probability that at least one input y satisfies $H(y) = H(x)$ is 0.5?

For a single value of y , the probability that $H(y) = H(x)$ is just $1/n$. Conversely, the probability that $H(y) \neq H(x)$ is $[1 - (1/n)]$. If we generate k random values of y , then the probability that none of them match is just the product of the probabilities that each individual value does not match, or $[1 - (1/n)]^k$. Thus, the probability that there is at least one match is $1 - [1 - (1/n)]^k$.

The binomial theorem can be stated as

$$(1 - a)^k = 1 - ka + \frac{k(k-1)}{2!}a^2 - \frac{k(k-1)(k-2)}{3!}a^3 \dots$$

For very small values of a , this can be approximated as $(1 - ka)$. Thus, the probability of at least one match is approximated as $1 - [1 - (1/n)]^k \approx 1 - [1 - (k/n)] = k/n$. For a probability of 0.5, we have $k = n/2$.

In particular, for an m -bit hash code, the number of possible codes is 2^m and the value of k that produces a probability of one-half is

$$k = 2^{(m-1)} \quad (11.1)$$

The Birthday Paradox

The birthday paradox is often presented in elementary probability courses to demonstrate that probability results are sometimes counterintuitive. The problem can be stated as follows. What is the minimum value of k such that the probability is greater than 0.5 that at least two people in a group of k people have the same birthday? Ignore February 29 and assume that each birthday is equally likely.

We can reason to the answer as follows. The probability that the birthdays of any two people are not alike is clearly 364/365 (since there is only one chance in 365 that one person's birthday will coincide with another's). The probability that a third person's birthday will differ from the other two is 363/365; a fourth person's, 362/365; and so on, until we reach the 24th person (342/365). We thus obtain a series of 23 fractions which must be multiplied together to reach the probability that all 24 birthdays are different. The product is a fraction that reduces to about 0.507, or slightly better than 1/2, for a coincidence among 23 people.

To derive this answer formally, let us define

$$P(n, k) = \text{Pr[at least one duplicate in } k \text{ items, with each item able to take on one of } n \text{ equally likely values between 1 and } n]$$

Thus, we are looking for the smallest value of k such that $P(365, k) \geq 0.5$. It is easier first to derive the probability that there are no duplicates, which we designate as $Q(365, k)$. If $k > 365$, then it is impossible for all values to be different. So we assume $k \leq 365$. Now consider the number of different ways, N , that we can have k

values with no duplicates. We may choose any of the 365 values for the first item, any of the remaining 364 numbers for the second item, and so on. Hence, the number of different ways is

$$N = 365 \times 364 \times \dots (365 - k + 1) = \frac{365!}{(365 - k)!} \quad (11.2)$$

If we remove the restriction that there are no duplicates, then each item can be any of 365 values, and the total number of possibilities is 365^k . So the probability of no duplicates is simply the fraction of sets of values that have no duplicates out of all possible sets of values:

$$Q(365, k) = \frac{365!/(365 - k)!}{(365)^k} = \frac{365!}{(365 - k)!(365)^k}$$

and

$$P(365, k) = 1 - Q(365, k) = 1 - \frac{365!}{(365 - k)!(365)^k} \quad (11.3)$$

This function is plotted in Figure 11.13. The probabilities may seem surprisingly large to anyone who has not considered the problem before. Many people would guess that to have a probability greater than 0.5 that there is at least one duplicate, the number of people in the group would have to be about 100. In fact, the number is 23, with $P(365, 23) = 0.5073$. For $k = 100$, the probability of at least one duplicate is 0.9999997.

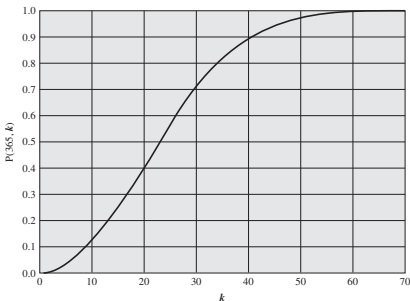


Figure 11.13 The Birthday Paradox

Perhaps the reason that the result seems so surprising is that if you consider a particular person in a group, the probability that some other person in the group has the same birthday is small. But the probability that we are concerned with is the probability that *any* pair of people in the group has the same birthday. In a group of 23, there are $(23(23 - 1))/2 = 253$ different pairs of people. Hence the high probabilities.

Useful Inequality

Before developing a generalization of the birthday problem, we derive an inequality that will be needed:

$$(1 - x) \leq e^{-x} \quad \text{for all } x \geq 0 \quad (11.4)$$

Figure 11.14 illustrates the inequality. To see that the inequality holds, note that the lower line is the tangent to e^{-x} at $x = 0$. The slope of that line is just the derivative of e^{-x} at $x = 0$:

$$\begin{aligned} f(x) &= e^{-x} \\ f'(x) &= \frac{d}{dx} e^{-x} = -e^{-x} \\ f'(0) &= -1 \end{aligned}$$

The tangent is a straight line of the form $ax + b$ with $a = -1$, and the tangent at $x = 0$ must equal $e^{-0} = 1$. Thus, the tangent is the function $(1 - x)$, confirming the inequality of Equation (11.4). Furthermore, note that for small x , we have $(1 - x) \approx e^{-x}$.

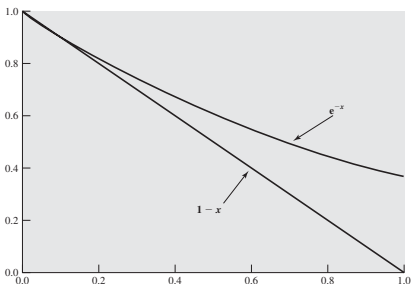


Figure 11.14 A Useful Inequality

The General Case of Duplications

The birthday problem can be generalized to the following problem. Given a random variable that is an integer with uniform distribution between 1 and n and a selection of k instances ($k \leq n$) of the random variable, what is the probability, $P(n, k)$, that there is at least one duplicate? The birthday problem is just the special case with $n = 365$. By the same reasoning as before, we have the following generalization of Equation (11.3):

$$P(n, k) = 1 - \frac{n!}{(n-k)!n^k} \quad (11.5)$$

We can rewrite this as

$$\begin{aligned} P(n, k) &= 1 - \frac{n \times (n-1) \times \cdots \times (n-k+1)}{n^k} \\ &= 1 - \left[\frac{n-1}{n} \times \frac{n-2}{n} \times \cdots \times \frac{n-k+1}{n} \right] \\ &= 1 - \left[\left(1 - \frac{1}{n}\right) \times \left(1 - \frac{2}{n}\right) \times \cdots \times \left(1 - \frac{k-1}{n}\right) \right] \end{aligned}$$

Using the inequality of Equation (11.4),

$$\begin{aligned} P(n, k) &> 1 - [(e^{-1/n}) \times (e^{-2/n}) \times \cdots \times (e^{-(k-1)/n})] \\ &> 1 - e^{-[(1/n) + (2/n) + \cdots + ((k-1)/n)]} \\ &> 1 - e^{-(k \times (k-1))/2n} \end{aligned}$$

Now let us pose the question: What value of k is required such that $P(n, k) > 0.5$? To satisfy the requirement, we have

$$\begin{aligned} 1/2 &= 1 - e^{-(k \times (k-1))/2n} \\ 2 &= e^{(k \times (k-1))/2n} \\ \ln 2 &= \frac{k \times (k-1)}{2n} \end{aligned}$$

For large k , we can replace $k \times (k-1)$ by k^2 , and we get

$$k = \sqrt{2(\ln 2)n} = 1.18\sqrt{n} \approx \sqrt{n} \quad (11.6)$$

As a reality check, for $n = 365$, we get $k = 1.18 \times \sqrt{365} = 22.54$, which is very close to the correct answer of 23.

We can now state the basis of the birthday attack in the following terms. Suppose we have a function H , with 2^m possible outputs (i.e., an m -bit output). If H is applied to k random inputs, what must be the value of k so that there is the probability of at least one duplicate [i.e., $H(x) = H(y)$ for some inputs x, y]? Using the approximation in Equation (11.6),

$$k = \sqrt{2^m} = 2^{m/2} \quad (11.7)$$

Overlap between Two Sets

There is a problem related to the general case of duplications that is also of relevance for our discussions. The problem is this: Given an integer random variable with uniform distribution between 1 and n and two sets of k instances ($k \leq n$) of the random variable, what is the probability, $R(n, k)$, that the two sets are not disjoint; that is, what is the probability that there is at least one value found in both sets?

Let us call the two sets X and Y , with elements $\{x_1, x_2, \dots, x_k\}$ and $\{y_1, y_2, \dots, y_k\}$, respectively. Given the value of x_1 , the probability that $y_1 = x_1$ is just $1/n$, and therefore the probability that y_1 does not match x_1 is $[1 - (1/n)]$. If we generate the k random values in Y , the probability that none of these values is equal to x_1 is $[1 - (1/n)]^k$. Thus, the probability that there is at least one match to x_1 is $1 - [1 - (1/n)]^k$.

To proceed, let us make the assumption that all the elements of X are distinct. If n is large and if k is also large (e.g., on the order of $\sqrt{n}$), then this is a good approximation. In fact, there may be a few duplications, but most of the values will be distinct. With that assumption, we can make the following derivation:

$$\begin{aligned}\Pr[\text{no match in } Y \text{ to } x_1] &= \left(1 - \frac{1}{n}\right)^k \\ \Pr[\text{no match in } Y \text{ to } X] &= \left(\left(1 - \frac{1}{n}\right)^k\right)^k = \left(1 - \frac{1}{n}\right)^{k^2} \\ R(n, k) &= \Pr[\text{at least one match in } Y \text{ to } X] = 1 - \left(1 - \frac{1}{n}\right)^{k^2}\end{aligned}$$

Using the inequality of Equation (11.4),

$$\begin{aligned}R(n, k) &> 1 - (e^{-1/n})^{k^2} \\ R(n, k) &> 1 - (e^{-k^2/n})\end{aligned}$$

Let us pose the question: What value of k is required such that $R(n, k) > 0.5$? To satisfy the requirement, we have

$$\begin{aligned}1/2 &= 1 - (e^{-k^2/n}) \\ 2 &= e^{k^2/n} \\ \ln(2) &= \frac{k^2}{n} \\ k &= \sqrt{(\ln(2))n} = 0.83\sqrt{n} \approx \sqrt{n}\end{aligned}\tag{11.8}$$

We can state this in terms related to birthday attacks as follows. Suppose we have a function H with 2^m possible outputs (i.e., an m -bit output). Apply H to k random inputs to produce the set X and again to k additional random inputs to produce the set Y . What must be the value of k so that there is the probability of at least 0.5 that there is a match between the two sets (i.e., $H(x) = H(y)$ for some inputs $x \in X, y \in Y$)? Using the approximation in Equation (11.8):

$$k = \sqrt{2^m} = 2^{m/2}$$

MESSAGE AUTHENTICATION CODES

12.1 Message Authentication Requirements**12.2 Message Authentication Functions**

Message Encryption

Message Authentication Code

12.3 Requirements for Message Authentication Codes**12.4 Security of MACs**

Brute-Force Attacks

Cryptanalysis

12.5 MACs Based on Hash Functions: HMAC

HMAC Design Objectives

HMAC Algorithm

Security of HMAC

12.6 MACs Based on Block Ciphers: DAA and CMAC

Data Authentication Algorithm

Cipher-Based Message Authentication Code (CMAC)

12.7 Authenticated Encryption: CCM and GCM

Counter with Cipher Block Chaining-Message Authentication Code

Galois/Counter Mode

12.8 Pseudorandom Number Generation Using Hash Functions and Macs

PRNG Based on Hash function

PRNG Based on MAC function

12.9 Recommended Reading and Web Site**12.10 Key Terms, Review Questions, and Problems**

At cats' green on the Sunday he took the message from the inside of the pillar and added Peter Moran's name to the two names already printed there in the "Brontosaur" code. The message now read: "Leviathan to Dragon: Martin Hillman, Trevor Allan, Peter Moran: observe and tail." What was the good of it John hardly knew. He felt better, he felt that at last he had made an attack on Peter Moran instead of waiting passively and effecting no retaliation. Besides, what was the use of being in possession of the key to the codes if he never took advantage of it?

—*Talking to Strange Men*, Ruth Rendell

KEY POINTS

- ◆ Message authentication is a mechanism or service used to verify the integrity of a message. Message authentication assures that data received are exactly as sent by (i.e., contain no modification, insertion, deletion, or replay) and that the purported identity of the sender is valid.
- ◆ Symmetric encryption provides authentication among those who share the secret key.
- ◆ A message authentication code (MAC) is an algorithm that requires the use of a secret key. A MAC takes a variable-length message and a secret key as input and produces an authentication code. A recipient in possession of the secret key can generate an authentication code to verify the integrity of the message.
- ◆ One means of forming a MAC is to combine a cryptographic hash function in some fashion with a secret key.
- ◆ Another approach to constructing a MAC is to use a symmetric block cipher in such a way that it produces a fixed-length output for a variable-length input.

One of the most fascinating and complex areas of cryptography is that of message authentication and the related area of digital signatures. It would be impossible, in anything less than book length, to exhaust all the cryptographic functions and protocols that have been proposed or implemented for message authentication and digital signatures. Instead, the purpose of this chapter and the next is to provide a broad overview of the subject and to develop a systematic means of describing the various approaches.

This chapter begins with an introduction to the requirements for authentication and digital signature and the types of attacks to be countered. Then the basic approaches are surveyed. The remainder of the chapter deals with the fundamental approach to message authentication known as the message authentication code (MAC). Following an overview of this topic, the chapter looks at security considerations for MACs. This is followed by a discussion of specific MACs in two categories: those built from cryptographic hash functions and those built using a block cipher

mode of operation. Next, we look at a relatively recent approach known as authenticated encryption. Finally, we look at the use of cryptographic hash functions and MACs for pseudorandom number generation.

12.1 MESSAGE AUTHENTICATION REQUIREMENTS

In the context of communications across a network, the following attacks can be identified.

1. **Disclosure:** Release of message contents to any person or process not possessing the appropriate cryptographic key.
2. **Traffic analysis:** Discovery of the pattern of traffic between parties. In a connection-oriented application, the frequency and duration of connections could be determined. In either a connection-oriented or connectionless environment, the number and length of messages between parties could be determined.
3. **Masquerade:** Insertion of messages into the network from a fraudulent source. This includes the creation of messages by an opponent that are purported to come from an authorized entity. Also included are fraudulent acknowledgments of message receipt or nonreceipt by someone other than the message recipient.
4. **Content modification:** Changes to the contents of a message, including insertion, deletion, transposition, and modification.
5. **Sequence modification:** Any modification to a sequence of messages between parties, including insertion, deletion, and reordering.
6. **Timing modification:** Delay or replay of messages. In a connection-oriented application, an entire session or sequence of messages could be a replay of some previous valid session, or individual messages in the sequence could be delayed or replayed. In a connectionless application, an individual message (e.g., datagram) could be delayed or replayed.
7. **Source repudiation:** Denial of transmission of message by source.
8. **Destination repudiation:** Denial of receipt of message by destination.

Measures to deal with the first two attacks are in the realm of message confidentiality and are dealt with in Part One. Measures to deal with items (3) through (6) in the foregoing list are generally regarded as message authentication. Mechanisms for dealing specifically with item (7) come under the heading of digital signatures. Generally, a digital signature technique will also counter some or all of the attacks listed under items (3) through (6). Dealing with item (8) may require a combination of the use of digital signatures and a protocol designed to counter this attack.

In summary, message authentication is a procedure to verify that received messages come from the alleged source and have not been altered. Message authentication may also verify sequencing and timeliness. A digital signature is an authentication technique that also includes measures to counter repudiation by the source.

12.2 MESSAGE AUTHENTICATION FUNCTIONS

Any message authentication or digital signature mechanism has two levels of functionality. At the lower level, there must be some sort of function that produces an authenticator: a value to be used to authenticate a message. This lower-level function is then used as a primitive in a higher-level authentication protocol that enables a receiver to verify the authenticity of a message.

This section is concerned with the types of functions that may be used to produce an authenticator. These may be grouped into three classes.

- **Hash function:** A function that maps a message of any length into a fixed-length hash value, which serves as the authenticator
- **Message encryption:** The ciphertext of the entire message serves as its authenticator
- **Message authentication code (MAC):** A function of the message and a secret key that produces a fixed-length value that serves as the authenticator

Hash functions, and how they may serve for message authentication, are discussed in Chapter 11. The remainder of this section briefly examines the remaining two topics. The remainder of the chapter elaborates on the topic of MACs.

Message Encryption

Message encryption by itself can provide a measure of authentication. The analysis differs for symmetric and public-key encryption schemes.

SYMMETRIC ENCRYPTION Consider the straightforward use of symmetric encryption (Figure 12.1a). A message M transmitted from source A to destination B is encrypted using a secret key K shared by A and B. If no other party knows the key, then confidentiality is provided: No other party can recover the plaintext of the message.

In addition, B is assured that the message was generated by A. Why? The message must have come from A, because A is the only other party that possesses K and therefore the only other party with the information necessary to construct ciphertext that can be decrypted with K . Furthermore, if M is recovered, B knows that none of the bits of M have been altered, because an opponent that does not know K would not know how to alter bits in the ciphertext to produce the desired changes in the plaintext.

So we may say that symmetric encryption provides authentication as well as confidentiality. However, this flat statement needs to be qualified. Consider exactly what is happening at B. Given a decryption function D and a secret key K , the destination will accept any input X and produce output $Y = D(K, X)$. If X is the ciphertext of a legitimate message M produced by the corresponding encryption function, then Y is some plaintext message M . Otherwise, Y will likely be a meaningless sequence of bits. There may need to be some automated means of determining at B whether Y is legitimate plaintext and therefore must have come from A.

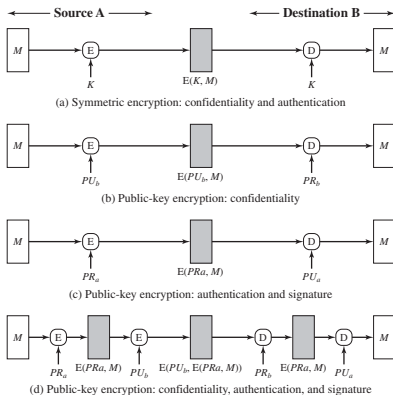


Figure 12.1 Basic Uses of Message Encryption

The implications of the line of reasoning in the preceding paragraph are profound from the point of view of authentication. Suppose the message M can be any arbitrary bit pattern. In that case, there is no way to determine automatically, at the destination, whether an incoming message is the ciphertext of a legitimate message. This conclusion is incontrovertible: If M can be any bit pattern, then regardless of the value of X , the value $Y = D(K, X)$ is *some* bit pattern and therefore must be accepted as authentic plaintext.

Thus, in general, we require that only a small subset of all possible bit patterns be considered legitimate plaintext. In that case, any spurious ciphertext is unlikely to produce legitimate plaintext. For example, suppose that only one bit pattern in 10^6 is legitimate plaintext. Then the probability that any randomly chosen bit pattern, treated as ciphertext, will produce a legitimate plaintext message is only 10^{-6} .

For a number of applications and encryption schemes, the desired conditions prevail as a matter of course. For example, suppose that we are transmitting English-language messages using a Caesar cipher with a shift of one ($K = 1$). A sends the following legitimate ciphertext:

nbsftfbupbutboeepftfbupbutboemjumfmbnctfbujwz

B decrypts to produce the following plaintext:

mareseatoatsanddoesatoatsandlittlelambseativy

A simple frequency analysis confirms that this message has the profile of ordinary English. On the other hand, if an opponent generates the following random sequence of letters:

zuvrsoevgqxlzwigamdvnmhpmccxiuureosfbcebtqxsxq

this decrypts to

ytugrndufpwyvhfzlcumlgolbbwhhttqdnreabdasprwvp

which does not fit the profile of ordinary English.

It may be difficult to determine *automatically* if incoming ciphertext decrypts to intelligible plaintext. If the plaintext is, say, a binary object file or digitized X-rays, determination of properly formed and therefore authentic plaintext may be difficult. Thus, an opponent could achieve a certain level of disruption simply by issuing messages with random content purporting to come from a legitimate user.

One solution to this problem is to force the plaintext to have some structure that is easily recognized but that cannot be replicated without recourse to the encryption function. We could, for example, append an error-detecting code, also known as a frame check sequence (FCS) or checksum, to each message before encryption, as illustrated in Figure 12.2a. A prepares a plaintext message M and then provides this as input to a function F that produces an FCS. The FCS is appended to M and the entire block is then encrypted. At the destination, B

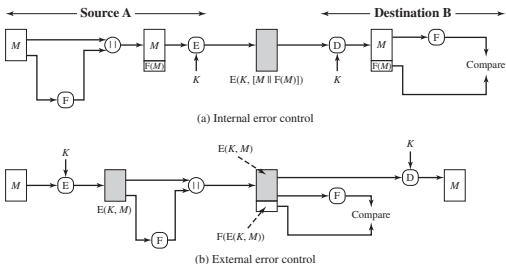


Figure 12.2 Internal and External Error Control

decrypts the incoming block and treats the results as a message with an appended FCS. B applies the same function F to attempt to reproduce the FCS. If the calculated FCS is equal to the incoming FCS, then the message is considered authentic. It is unlikely that any random sequence of bits would exhibit the desired relationship.

Note that the order in which the FCS and encryption functions are performed is critical. The sequence illustrated in Figure 12.2a is referred to in [DIFF79] as **internal error control**, which the authors contrast with **external error control** (Figure 12.2b). With internal error control, authentication is provided because an opponent would have difficulty generating ciphertext that, when decrypted, would have valid error control bits. If instead the FCS is the outer code, an opponent can construct messages with valid error-control codes. Although the opponent cannot know what the decrypted plaintext will be, he or she can still hope to create confusion and disrupt operations.

An error-control code is just one example; in fact, any sort of structuring added to the transmitted message serves to strengthen the authentication capability. Such structure is provided by the use of a communications architecture consisting of layered protocols. As an example, consider the structure of messages transmitted using the TCP/IP protocol architecture. Figure 12.3 shows the format of a TCP segment, illustrating the TCP header. Now suppose that each pair of hosts shared a unique secret key, so that all exchanges between a pair of hosts used the same key, regardless of application. Then we could simply encrypt all of the datagram except the IP header. Again, if an opponent substituted some arbitrary bit pattern for the encrypted TCP segment, the resulting plaintext would not include a meaningful header. In this case, the header includes not only a checksum (which covers the header) but also other useful information, such as the sequence number. Because successive TCP segments on a given connection are numbered sequentially, encryption assures that an opponent does not delay, misorder, or delete any segments.

PUBLIC-KEY ENCRYPTION The straightforward use of public-key encryption (Figure 12.1b) provides confidentiality but not authentication. The source (A) uses the public key PU_b of the destination (B) to encrypt M . Because only B has the

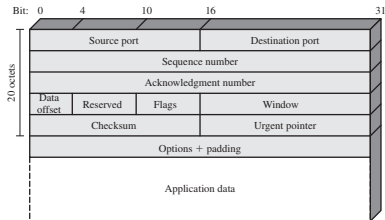


Figure 12.3 TCP Segment

corresponding private key PR_b , only B can decrypt the message. This scheme provides no authentication, because any opponent could also use B's public key to encrypt a message and claim to be A.

To provide authentication, A uses its private key to encrypt the message, and B uses A's public key to decrypt (Figure 12.1c). This provides authentication using the same type of reasoning as in the symmetric encryption case: The message must have come from A because A is the only party that possesses PR_a and therefore the only party with the information necessary to construct ciphertext that can be decrypted with PU_a . Again, the same reasoning as before applies: There must be some internal structure to the plaintext so that the receiver can distinguish between well-formed plaintext and random bits.

Assuming there is such structure, then the scheme of Figure 12.1c does provide authentication. It also provides what is known as digital signature.¹ Only A could have constructed the ciphertext because only A possesses PR_a . Not even B, the recipient, could have constructed the ciphertext. Therefore, if B is in possession of the ciphertext, B has the means to prove that the message must have come from A. In effect, A has “signed” the message by using its private key to encrypt. Note that this scheme does not provide confidentiality. Anyone in possession of A's public key can decrypt the ciphertext.

To provide both confidentiality and authentication, A can encrypt M first using its private key, which provides the digital signature, and then using B's public key, which provides confidentiality (Figure 12.1d). The disadvantage of this approach is that the public-key algorithm, which is complex, must be exercised four times rather than two in each communication.

Message Authentication Code

An alternative authentication technique involves the use of a secret key to generate a small fixed-size block of data, known as a **cryptographic checksum** or MAC, that is appended to the message. This technique assumes that two communicating parties, say A and B, share a common secret key K . When A has a message to send to B, it calculates the MAC as a function of the message and the key:

$$\text{MAC} = C(K, M)$$

where

M = input message

C = MAC function

K = shared secret key

MAC = message authentication code

The message plus MAC are transmitted to the intended recipient. The recipient performs the same calculation on the received message, using the same secret key, to

¹This is not the way in which digital signatures are constructed, as we shall see, but the principle is the same.

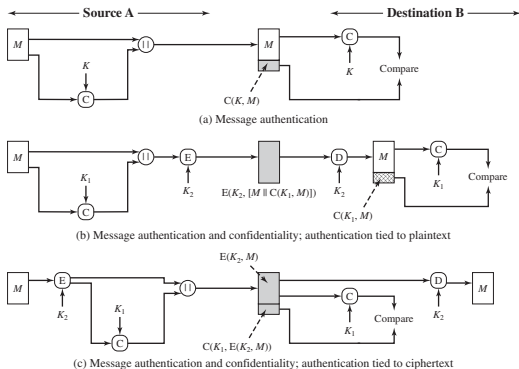


Figure 12.4 Basic Uses of Message Authentication code (MAC)

generate a new MAC. The received MAC is compared to the calculated MAC (Figure 12.4a). If we assume that only the receiver and the sender know the identity of the secret key, and if the received MAC matches the calculated MAC, then

1. The receiver is assured that the message has not been altered. If an attacker alters the message but does not alter the MAC, then the receiver's calculation of the MAC will differ from the received MAC. Because the attacker is assumed not to know the secret key, the attacker cannot alter the MAC to correspond to the alterations in the message.
2. The receiver is assured that the message is from the alleged sender. Because no one else knows the secret key, no one else could prepare a message with a proper MAC.
3. If the message includes a sequence number (such as is used with HDLC, X.25, and TCP), then the receiver can be assured of the proper sequence because an attacker cannot successfully alter the sequence number.

A MAC function is similar to encryption. One difference is that the MAC algorithm need not be reversible, as it must be for decryption. In general, the MAC function is a many-to-one function. The domain of the function consists of messages of some arbitrary length, whereas the range consists of all possible MACs and all possible keys. If an n -bit MAC is used, then there are 2^n possible MACs, whereas

there are N possible messages with $N \gg 2^n$. Furthermore, with a k -bit key, there are 2^k possible keys.

For example, suppose that we are using 100-bit messages and a 10-bit MAC. Then, there are a total of 2^{100} different messages but only 2^{10} different MACs. So, on average, each MAC value is generated by a total of $2^{100}/2^{10} = 2^{90}$ different messages. If a 5-bit key is used, then there are $2^5 = 32$ different mappings from the set of messages to the set of MAC values.

It turns out that, because of the mathematical properties of the authentication function, it is less vulnerable to being broken than encryption.

The process depicted in Figure 12.4a provides authentication but not confidentiality, because the message as a whole is transmitted in the clear. Confidentiality can be provided by performing message encryption either after (Figure 12.4b) or before (Figure 12.4c) the MAC algorithm. In both these cases, two separate keys are needed, each of which is shared by the sender and the receiver. In the first case, the MAC is calculated with the message as input and is then concatenated to the message. The entire block is then encrypted. In the second case, the message is encrypted first. Then the MAC is calculated using the resulting ciphertext and is concatenated to the ciphertext to form the transmitted block. Typically, it is preferable to tie the authentication directly to the plaintext, so the method of Figure 12.4b is used.

Because symmetric encryption will provide authentication and because it is widely used with readily available products, why not simply use this instead of a separate message authentication code? [DAVI89] suggests three situations in which a message authentication code is used.

1. There are a number of applications in which the same message is broadcast to a number of destinations. Examples are notification to users that the network is now unavailable or an alarm signal in a military control center. It is cheaper and more reliable to have only one destination responsible for monitoring authenticity. Thus, the message must be broadcast in plaintext with an associated message authentication code. The responsible system has the secret key and performs authentication. If a violation occurs, the other destination systems are alerted by a general alarm.
2. Another possible scenario is an exchange in which one side has a heavy load and cannot afford the time to decrypt all incoming messages. Authentication is carried out on a selective basis, messages being chosen at random for checking.
3. Authentication of a computer program in plaintext is an attractive service. The computer program can be executed without having to decrypt it every time, which would be wasteful of processor resources. However, if a message authentication code were attached to the program, it could be checked whenever assurance was required of the integrity of the program.

Three other rationales may be added.

4. For some applications, it may not be of concern to keep messages secret, but it is important to authenticate messages. An example is the Simple Network Management Protocol Version 3 (SNMPv3), which separates the functions of

confidentiality and authentication. For this application, it is usually important for a managed system to authenticate incoming SNMP messages, particularly if the message contains a command to change parameters at the managed system. On the other hand, it may not be necessary to conceal the SNMP traffic.

5. Separation of authentication and confidentiality functions affords architectural flexibility. For example, it may be desired to perform authentication at the application level but to provide confidentiality at a lower level, such as the transport layer.
6. A user may wish to prolong the period of protection beyond the time of reception and yet allow processing of message contents. With message encryption, the protection is lost when the message is decrypted, so the message is protected against fraudulent modifications only in transit but not within the target system.

Finally, note that the MAC does not provide a digital signature, because both sender and receiver share the same key.

12.3 REQUIREMENTS FOR MESSAGE AUTHENTICATION CODES

A MAC, also known as a cryptographic checksum, is generated by a function C of the form

$$T = \text{MAC}(K, M)$$

where M is a variable-length message, K is a secret key shared only by sender and receiver, and $\text{MAC}(K, M)$ is the fixed-length authenticator, sometimes called a **tag**. The tag is appended to the message at the source at a time when the message is assumed or known to be correct. The receiver authenticates that message by recomputing the tag.

When an entire message is encrypted for confidentiality, using either symmetric or asymmetric encryption, the security of the scheme generally depends on the bit length of the key. Barring some weakness in the algorithm, the opponent must resort to a brute-force attack using all possible keys. On average, such an attack will require $2^{(k-1)}$ attempts for a k -bit key. In particular, for a ciphertext-only attack, the opponent, given ciphertext C , performs $P_i = D(K_i, C)$ for all possible key values K_i until a P_i is produced that matches the form of acceptable plaintext.

In the case of a MAC, the considerations are entirely different. In general, the MAC function is a many-to-one function, due to the many-to-one nature of the function. Using brute-force methods, how would an opponent attempt to discover a key? If confidentiality is not employed, the opponent has access to plaintext messages and their associated MACs. Suppose $k > n$; that is, suppose that the key size is greater than the MAC size. Then, given a known M_1 and T_1 , with $T_1 = \text{MAC}(K, M_1)$, the cryptanalyst can perform $T_i = \text{MAC}(K_i, M_1)$ for all possible key values K_i . At least one key is guaranteed to produce a match of $T_i = T_1$. Note that a total of 2^k tags will be produced, but there are only $2^n < 2^k$ different tag values. Thus, a number of keys will produce the correct tag and the opponent has no

way of knowing which is the correct key. On average, a total of $2^k/2^n = 2^{(k-n)}$ keys will produce a match. Thus, the opponent must iterate the attack.

• **Round 1**

Given: $M_1, T_1 = \text{MAC}(K, M_1)$
 Compute $T_i = \text{MAC}(K_i, M_1)$ for all 2^k keys
 Number of matches $\approx 2^{(k-n)}$

• **Round 2**

Given: $M_2, T_2 = \text{MAC}(K, M_2)$
 Compute $T_i = \text{MAC}(K_i, M_2)$ for the $2^{(k-n)}$ keys resulting from Round 1
 Number of matches $\approx 2^{(k-2 \times n)}$

And so on. On average, α rounds will be needed if $k = \alpha \times n$. For example, if an 80-bit key is used and the tag is 32 bits, then the first round will produce about 2^{48} possible keys. The second round will narrow the possible keys to about 2^{16} possibilities. The third round should produce only a single key, which must be the one used by the sender.

If the key length is less than or equal to the tag length, then it is likely that a first round will produce a single match. It is possible that more than one key will produce such a match, in which case the opponent would need to perform the same test on a new (message, tag) pair.

Thus, a brute-force attempt to discover the authentication key is no less effort and may be more effort than that required to discover a decryption key of the same length. However, other attacks that do not require the discovery of the key are possible.

Consider the following MAC algorithm. Let $M = (X_1 \parallel X_2 \parallel \dots \parallel X_m)$ be a message that is treated as a concatenation of 64-bit blocks X_i . Then define

$$\Delta(M) = X_1 \oplus X_2 \oplus \dots \oplus X_m$$

$$\text{MAC}(K, M) = E(K, \Delta(M))$$

where $\oplus$ is the exclusive-OR (XOR) operation and the encryption algorithm is DES in electronic codebook mode. Thus, the key length is 56 bits, and the tag length is 64 bits. If an opponent observes $\{M \parallel \text{MAC}(K, M)\}$, a brute-force attempt to determine K will require at least 2^{56} encryptions. But the opponent can attack the system by replacing X_1 through X_{m-1} with any desired values Y_1 through Y_{m-1} and replacing X_m with Y_m , where Y_m is calculated as

$$Y_m = Y_1 \oplus Y_2 \oplus \dots \oplus Y_{m-1} \oplus \Delta(M)$$

The opponent can now concatenate the new message, which consists of Y_1 through Y_m , using the original tag to form a message that will be accepted as authentic by the receiver. With this tactic, any message of length $64 \times (m - 1)$ bits can be fraudulently inserted.

Thus, in assessing the security of a MAC function, we need to consider the types of attacks that may be mounted against it. With that in mind, let us state the requirements for the function. Assume that an opponent knows the MAC function

but does not know K . Then the MAC function should satisfy the following requirements.

1. If an opponent observes M and $\text{MAC}(K, M)$, it should be computationally infeasible for the opponent to construct a message M' such that

$$\text{MAC}(K, M') = \text{MAC}(K, M)$$

2. $\text{MAC}(K, M)$ should be uniformly distributed in the sense that for randomly chosen messages, M and M' , the probability that $\text{MAC}(K, M) = \text{MAC}(K, M')$ is 2^{-n} , where n is the number of bits in the tag.
3. Let M' be equal to some known transformation on M . That is, $M' = f(M)$. For example, f may involve inverting one or more specific bits. In that case,

$$\Pr[\text{MAC}(K, M) = \text{MAC}(K, M')] = 2^{-n}$$

The first requirement speaks to the earlier example, in which an opponent is able to construct a new message to match a given tag, even though the opponent does not know and does not learn the key. The second requirement deals with the need to thwart a brute-force attack based on chosen plaintext. That is, if we assume that the opponent does not know K but does have access to the MAC function and can present messages for MAC generation, then the opponent could try various messages until finding one that matches a given tag. If the MAC function exhibits uniform distribution, then a brute-force method would require, on average, $2^{(n-1)}$ attempts before finding a message that fits a given tag.

The final requirement dictates that the authentication algorithm should not be weaker with respect to certain parts or bits of the message than others. If this were not the case, then an opponent who had M and $\text{MAC}(K, M)$ could attempt variations on M at the known “weak spots” with a likelihood of early success at producing a new message that matched the old tags.

12.4 SECURITY OF MACS

Just as with encryption algorithms and hash functions, we can group attacks on MACs into two categories: brute-force attacks and cryptanalysis.

Brute-Force Attacks

A brute-force attack on a MAC is a more difficult undertaking than a brute-force attack on a hash function because it requires known message-tag pairs. Let us see why this is so. To attack a hash code, we can proceed in the following way. Given a fixed message x with n -bit hash code $h = H(x)$, a brute-force method of finding a collision is to pick a random bit string y and check if $H(y) = H(x)$. The attacker can do this repeatedly off line. Whether an off-line attack can be used on a MAC algorithm depends on the relative size of the key and the tag.

To proceed, we need to state the desired security property of a MAC algorithm, which can be expressed as follows.

- **Computation resistance:** Given one or more text-MAC pairs $[x_i, \text{MAC}(K, x_i)]$, it is computationally infeasible to compute any text-MAC pair $[x, \text{MAC}(K, x)]$ for any new input $x \neq x_i$.

In other words, the attacker would like to come up with the valid MAC code for a given message x . There are two lines of attack possible: attack the key space and attack the MAC value. We examine each of these in turn.

If an attacker can determine the MAC key, then it is possible to generate a valid MAC value for any input x . Suppose the key size is k bits and that the attacker has one known text–tag pair. Then the attacker can compute the n -bit tag on the known text for all possible keys. At least one key is guaranteed to produce the correct tag, namely, the valid key that was initially used to produce the known text–tag pair. This phase of the attack takes a level of effort proportional to 2^k (that is, one operation for each of the 2^k possible key values). However, as was described earlier, because the MAC is a many-to-one mapping, there may be other keys that produce the correct value. Thus, if more than one key is found to produce the correct value, additional text–tag pairs must be tested. It can be shown that the level of effort drops off rapidly with each additional text–MAC pair and that the overall level of effort is roughly 2^k [MENE97].

An attacker can also work on the tag without attempting to recover the key. Here, the objective is to generate a valid tag for a given message or to find a message that matches a given tag. In either case, the level of effort is comparable to that for attacking the one-way or weak collision-resistant property of a hash code, or 2^n . In the case of the MAC, the attack cannot be conducted off line without further input; the attacker will require chosen text–tag pairs or knowledge of the key.

To summarize, the level of effort for brute-force attack on a MAC algorithm can be expressed as $\min(2^k, 2^n)$. The assessment of strength is similar to that for symmetric encryption algorithms. It would appear reasonable to require that the key length and tag length satisfy a relationship such as $\min(k, n) \geq N$, where N is perhaps in the range of 128 bits.

Cryptanalysis

As with encryption algorithms and hash functions, cryptanalytic attacks on MAC algorithms seek to exploit some property of the algorithm to perform some attack other than an exhaustive search. The way to measure the resistance of a MAC algorithm to cryptanalysis is to compare its strength to the effort required for a brute-force attack. That is, an ideal MAC algorithm will require a cryptanalytic effort greater than or equal to the brute-force effort.

There is much more variety in the structure of MACs than in hash functions, so it is difficult to generalize about the cryptanalysis of MACs. Furthermore, far less work has been done on developing such attacks. A useful survey of some methods for specific MACs is [PREN96].

12.5 MACs BASED ON HASH FUNCTIONS: HMAC

Later in this chapter, we look at examples of a MAC based on the use of a symmetric block cipher. This has traditionally been the most common approach to constructing a MAC. In recent years, there has been increased interest in developing

a MAC derived from a cryptographic hash function. The motivations for this interest are

1. Cryptographic hash functions such as MD5 and SHA generally execute faster in software than symmetric block ciphers such as DES.
2. Library code for cryptographic hash functions is widely available.

With the development of AES and the more widespread availability of code for encryption algorithms, these considerations are less significant, but hash-based MACs continue to be widely used.

A hash function such as SHA was not designed for use as a MAC and cannot be used directly for that purpose, because it does not rely on a secret key. There have been a number of proposals for the incorporation of a secret key into an existing hash algorithm. The approach that has received the most support is HMAC [BELL96a, BELL96b]. HMAC has been issued as RFC 2104, has been chosen as the mandatory-to-implement MAC for IP security, and is used in other Internet protocols, such as SSL. HMAC has also been issued as a NIST standard (FIPS 198).

HMAC Design Objectives

RFC 2104 lists the following design objectives for HMAC.

- To use, without modifications, available hash functions. In particular, to use hash functions that perform well in software and for which code is freely and widely available.
- To allow for easy replaceability of the embedded hash function in case faster or more secure hash functions are found or required.
- To preserve the original performance of the hash function without incurring a significant degradation.
- To use and handle keys in a simple way.
- To have a well understood cryptographic analysis of the strength of the authentication mechanism based on reasonable assumptions about the embedded hash function.

The first two objectives are important to the acceptability of HMAC. HMAC treats the hash function as a “black box.” This has two benefits. First, an existing implementation of a hash function can be used as a module in implementing HMAC. In this way, the bulk of the HMAC code is prepackaged and ready to use without modification. Second, if it is ever desired to replace a given hash function in an HMAC implementation, all that is required is to remove the existing hash function module and drop in the new module. This could be done if a faster hash function were desired. More important, if the security of the embedded hash function were compromised, the security of HMAC could be retained simply by replacing the embedded hash function with a more secure one (e.g., replacing SHA-2 with SHA-3).

The last design objective in the preceding list is, in fact, the main advantage of HMAC over other proposed hash-based schemes. HMAC can be proven secure

provided that the embedded hash function has some reasonable cryptographic strengths. We return to this point later in this section, but first we examine the structure of HMAC.

HMAC Algorithm

Figure 12.5 illustrates the overall operation of HMAC. Define the following terms.

H = embedded hash function (e.g., MD5, SHA-1, RIPEMD-160)

IV = initial value input to hash function

M = message input to HMAC (including the padding specified in the embedded hash function)

Y_i = i th block of M , $0 \leq i \leq (L-1)$

L = number of blocks in M

b = number of bits in a block

n = length of hash code produced by embedded hash function

K = secret key; recommended length is $\geq n$; if key length is greater than b , the key is input to the hash function to produce an n -bit key

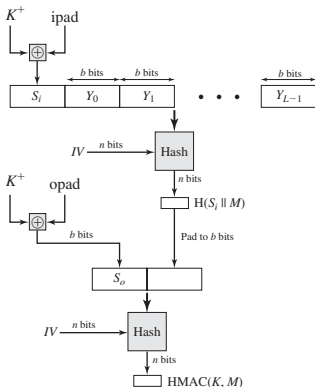


Figure 12.5 HMAC Structure

$K^+ = K$ padded with zeros on the left so that the result is b bits in length

ipad = 00110110 (36 in hexadecimal) repeated $b/8$ times

opad = 01011100 (5C in hexadecimal) repeated $b/8$ times

Then HMAC can be expressed as

$$\text{HMAC}(K, M) = H[(K^+ \oplus \text{opad}) \parallel H[(K^+ \oplus \text{ipad}) \parallel M]]$$

We can describe the algorithm as follows.

1. Append zeros to the left end of K to create a b -bit string K^+ (e.g., if K is of length 160 bits and $b = 512$, then K will be appended with 44 zeroes).
2. XOR (bitwise exclusive-OR) K^+ with ipad to produce the b -bit block S_i .
3. Append M to S_i .
4. Apply H to the stream generated in step 3.
5. XOR K^+ with opad to produce the b -bit block S_o .
6. Append the hash result from step 4 to S_o .
7. Apply H to the stream generated in step 6 and output the result.

Note that the XOR with ipad results in flipping one-half of the bits of K . Similarly, the XOR with opad results in flipping one-half of the bits of K , using a different set of bits. In effect, by passing S_i and S_o through the compression function of the hash algorithm, we have pseudorandomly generated two keys from K .

HMAC should execute in approximately the same time as the embedded hash function for long messages. HMAC adds three executions of the hash compression function (for S_i , S_o , and the block produced from the inner hash).

A more efficient implementation is possible, as shown in Figure 12.6. Two quantities are precomputed:

$$f(IV, (K^+ \oplus \text{ipad}))$$

$$f(IV, (K^+ \oplus \text{opad}))$$

where $f(\text{cv}, \text{block})$ is the compression function for the hash function, which takes as arguments a chaining variable of n bits and a block of b bits and produces a chaining variable of n bits. These quantities only need to be computed initially and every time the key changes. In effect, the precomputed quantities substitute for the initial value (IV) in the hash function. With this implementation, only one additional instance of the compression function is added to the processing normally produced by the hash function. This more efficient implementation is especially worthwhile if most of the messages for which a MAC is computed are short.

Security of HMAC

The security of any MAC function based on an embedded hash function depends in some way on the cryptographic strength of the underlying hash function. The appeal of HMAC is that its designers have been able to prove an exact relationship between the strength of the embedded hash function and the strength of HMAC.

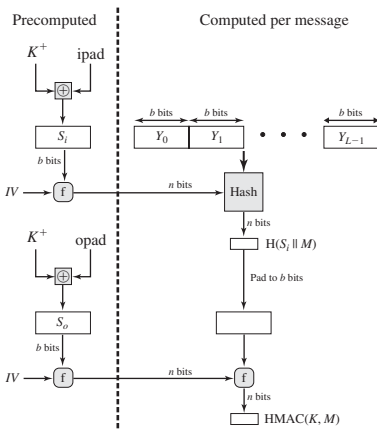


Figure 12.6 Efficient Implementation of HMAC

The security of a MAC function is generally expressed in terms of the probability of successful forgery with a given amount of time spent by the forger and a given number of message-tag pairs created with the same key. In essence, it is proved in [BELL96a] that for a given level of effort (time, message-tag pairs) on messages generated by a legitimate user and seen by the attacker, the probability of successful attack on HMAC is equivalent to one of the following attacks on the embedded hash function.

1. The attacker is able to compute an output of the compression function even with an IV that is random, secret, and unknown to the attacker.
2. The attacker finds collisions in the hash function even when the IV is random and secret.

In the first attack, we can view the compression function as equivalent to the hash function applied to a message consisting of a single b -bit block. For this attack, the IV of the hash function is replaced by a secret, random value of n bits. An attack on this hash function requires either a brute-force attack on the key, which is a level of effort on the order of 2^n , or a birthday attack, which is a special case of the second attack, discussed next.

In the second attack, the attacker is looking for two messages M and M' that produce the same hash: $H(M) = H(M')$. This is the birthday attack discussed in Chapter 11. We have shown that this requires a level of effort of $2^{n/2}$ for a hash length of n . On this basis, the security of MD5 is called into question, because a level of effort of 2^{64} looks feasible with today's technology. Does this mean that a 128-bit hash function such as MD5 is unsuitable for HMAC? The answer is no, because of the following argument. To attack MD5, the attacker can choose any set of messages and work on these off line on a dedicated computing facility to find a collision. Because the attacker knows the hash algorithm and the default IV , the attacker can generate the hash code for each of the messages that the attacker generates. However, when attacking HMAC, the attacker cannot generate message/code pairs off line because the attacker does not know K . Therefore, the attacker must observe a sequence of messages generated by HMAC under the same key and perform the attack on these known messages. For a hash code length of 128 bits, this requires 2^{64} observed blocks (2^{72} bits) generated using the same key. On a 1-Gbps link, one would need to observe a continuous stream of messages with no change in key for about 150,000 years in order to succeed. Thus, if speed is a concern, it is fully acceptable to use MD5 rather than SHA-1 as the embedded hash function for HMAC.

12.6 MACs BASED ON BLOCK CIPHERS: DAA AND CMAC

In this section, we look at two MACs that are based on the use of a block cipher mode of operation. We begin with an older algorithm, the Data Authentication Algorithm (DAA), which is now obsolete. Then we examine CMAC, which is designed to overcome the deficiencies of DAA.

Data Authentication Algorithm

The **Data Authentication Algorithm** (DAA), based on DES, has been one of the most widely used MACs for a number of years. The algorithm is both a FIPS publication (FIPS PUB 113) and an ANSI standard (X9.17). However, as we discuss subsequently, security weaknesses in this algorithm have been discovered, and it is being replaced by newer and stronger algorithms.

The algorithm can be defined as using the cipher block chaining (CBC) mode of operation of DES (Figure 6.4) with an initialization vector of zero. The data (e.g., message, record, file, or program) to be authenticated are grouped into contiguous 64-bit blocks: $D_1, D_2, \dots, D_N$. If necessary, the final block is padded on the right with zeroes to form a full 64-bit block. Using the DES encryption algorithm E and a secret key K , a data authentication code (DAC) is calculated as follows (Figure 12.7).

$$\begin{aligned} O_1 &= E(K, D) \\ O_2 &= E(K, [D_2 \oplus O_1]) \\ O_3 &= E(K, [D_3 \oplus O_2]) \\ &\vdots \\ &\vdots \\ O_N &= E(K, [D_N \oplus O_{N-1}]) \end{aligned}$$

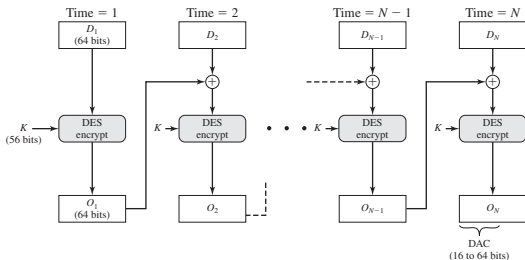


Figure 12.7 Data Authentication Algorithm (FIPS PUB 113)

The DAC consists of either the entire block O_N or the leftmost M bits of the block, with $16 \leq M \leq 64$.

Cipher-Based Message Authentication Code (CMAC)

As was mentioned, DAA has been widely adopted in government and industry. [BELL00] demonstrated that this MAC is secure under a reasonable set of security criteria, with the following restriction. Only messages of one fixed length of mn bits are processed, where n is the cipher block size and m is a fixed positive integer. As a simple example, notice that given the CBC MAC of a one-block message X , say $T = \text{MAC}(K, X)$, the adversary immediately knows the CBC MAC for the two-block message $X \parallel (X \oplus T)$ since this is once again T .

Black and Rogaway [BLAC00] demonstrated that this limitation could be overcome using three keys: one key of length K to be used at each step of the cipher block chaining and two keys of length n , where k is the key length and n is the cipher block length. This proposed construction was refined by Iwata and Kurosawa so that the two n -bit keys could be derived from the encryption key, rather than being provided separately [IWAT03]. This refinement, adopted by NIST, is the **Cipher-based Message Authentication Code (CMAC)** mode of operation for use with AES and triple DES. It is specified in NIST Special Publication 800-38B.

First, let us define the operation of CMAC when the message is an integer multiple n of the cipher block length b . For AES, $b = 128$, and for triple DES, $b = 64$. The message is divided into n blocks $(M_1, M_2, \dots, M_n)$. The algorithm makes use of a k -bit encryption key K and an n -bit constant, K_1 . For AES, the key size k is 128, 192, or 256 bits; for triple DES, the key size is 112 or 168 bits. CMAC is calculated as follows (Figure 12.8).

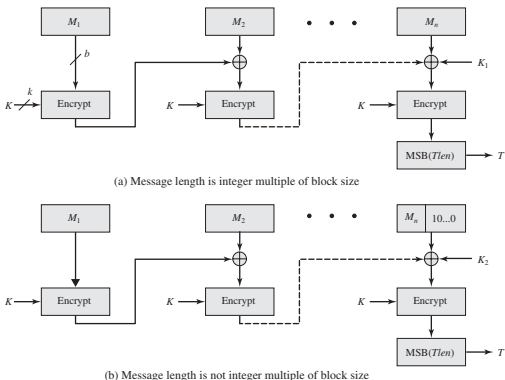


Figure 12.8 Cipher-Based Message Authentication Code (CMAC)

$$\begin{aligned}
 C_1 &= E(K, M_1) \\
 C_2 &= E(K, [M_2 \oplus C_1]) \\
 C_3 &= E(K, [M_3 \oplus C_2]) \\
 &\vdots \\
 C_n &= E(K, [M_n \oplus C_{n-1} \oplus K_1]) \\
 T &= \text{MSB}_{Tlen}(C_n)
 \end{aligned}$$

where

T = message authentication code, also referred to as the tag
 $Tlen$ = bit length of T
 $\text{MSB}_s(X)$ = the s leftmost bits of the bit string X

If the message is not an integer multiple of the cipher block length, then the final block is padded to the right (least significant bits) with a 1 and as many 0s as necessary so that the final block is also of length b . The CMAC operation then proceeds as before, except that a different n -bit key K_2 is used instead of K_1 .

The two n -bit keys are derived from the k -bit encryption key as follows.

$$\begin{aligned}L &= E(K, 0^n) \\ K_1 &= L \cdot x \\ K_2 &= L \cdot x^2 = (L \cdot x) \cdot x\end{aligned}$$

where multiplication ($\cdot$) is done in the finite field $GF(2^n)$ and x and x^2 are first- and second-order polynomials that are elements of $GF(2^n)$. Thus, the binary representation of x consists of $n - 2$ zeros followed by 10; the binary representation of x^2 consists of $n - 3$ zeros followed by 100. The finite field is defined with respect to an irreducible polynomial that is lexicographically first among all such polynomials with the minimum possible number of nonzero terms. For the two approved block sizes, the polynomials are $x^{64} + x^4 + x^3 + x + 1$ and $x^{128} + x^7 + x^2 + x + 1$.

To generate K_1 and K_2 , the block cipher is applied to the block that consists entirely of 0 bits. The first subkey is derived from the resulting ciphertext by a left shift of one bit and, conditionally, by XORing a constant that depends on the block size. The second subkey is derived in the same manner from the first subkey. This property of finite fields of the form $GF(2^n)$ was explained in the discussion of MixColumns in Chapter 5.

12.7 AUTHENTICATED ENCRYPTION: CCM AND GCM

Authenticated encryption (AE) is a term used to describe encryption systems that simultaneously protect confidentiality and authenticity (integrity) of communications. Many applications and protocols require both forms of security, but until recently the two services have been designed separately.

[BLAC05] discussed four common approaches to providing both confidentiality and encryption for a message M .

- **HtE: Hash-then-encrypt.** First compute the cryptographic hash function over M as $h = H(M)$. Then encrypt the message plus hash function: $E(K, (M||h))$.
- **MtE: MAC-then-encrypt.** Use two keys. First authenticate the plaintext by computing the MAC value as $T = \text{MAC}(K_1, M)$. Then encrypt the message plus tag: $E(K_2, (M || T))$. This approach is taken by the SSL/TLS protocols (Chapter 16).
- **EtM: Encrypt-then-MAC.** Use two keys. First encrypt the message to yield the ciphertext $C = E(K_2, M)$. Then authenticate the ciphertext with $T = \text{MAC}(K_1, C)$ to yield the pair (C, T) . This approach is used in the IPsec protocol (Chapter 19).
- **E&M: Encrypt-and-MAC.** Use two keys. Encrypt the message to yield the ciphertext $C = E(K_2, M)$. Authenticate the plaintext with $T = \text{MAC}(K_1, M)$ to yield the pair (C, T) . These operations can be performed in either order. This approach is used by the SSH protocol (Chapter 16).

Both decryption and verification are straightforward for each approach. For HtE, MtE, and E&M, decrypt first, then verify. For EtM, verify first, then decrypt. There are security vulnerabilities with all of these approaches. The HtE approach is

used in the Wired Equivalent Privacy (WEP) protocol to protect WiFi networks. This approach had fundamental weaknesses and led to the replacement of the WEP protocol. [BLAC05] and [BELL00] point out that there security concerns in each of the three encryption/MAC approaches listed above. Nevertheless, with proper design, any of these approaches can provide a high level of security. This is the goal of the two approaches discussed in this section, both of which have been standardized by NIST.

Counter with Cipher Block Chaining-Message Authentication Code

The CCM mode of operation was standardized by NIST specifically to support the security requirements of IEEE 802.11 WiFi wireless local area networks (Chapter 17), but can be used in any networking application requiring authenticated encryption. CCM is a variation of the encrypt-and-MAC approach to authenticated encryption. It is defined in NIST SP 800-38C.

The key algorithmic ingredients of CCM are the AES encryption algorithm (Chapter 5), the CTR mode of operation (Chapter 6), and the CMAC authentication algorithm (Section 12.6). A single key K is used for both encryption and MAC algorithms. The input to the CCM encryption process consists of three elements.

1. Data that will be both authenticated and encrypted. This is the plaintext message P of data block.
2. Associated data A that will be authenticated but not encrypted. An example is a protocol header that must be transmitted in the clear for proper protocol operation but which needs to be authenticated.
3. A nonce N that is assigned to the payload and the associated data. This is a unique value that is different for every instance during the lifetime of a protocol association and is intended to prevent replay attacks and certain other types of attacks.

Figure 12.9 illustrates the operation of CCM. For authentication, the input includes the nonce, the associated data, and the plaintext. This input is formatted as a sequence of blocks B_0 through B_r . The first block contains the nonce plus some formatting bits that indicate the lengths of the N , A , and P elements. This is followed by zero or more blocks that contain A , followed by zero or more blocks that contain P . The resulting sequence of blocks serves as input to the CMAC algorithm, which produces a MAC value with length $Tlen$, which is less than or equal to the block length (Figure 12.9a).

For encryption, a sequence of counters is generated that must be independent of the nonce. The authentication tag is encrypted in CTR mode using the single counter Ctr_0 . The $Tlen$ most significant bits of the output are XORed with the tag to produce an encrypted tag. The remaining counters are used for the CTR mode encryption of the plaintext (Figure 6.7). The encrypted plaintext is concatenated with the encrypted tag to form the ciphertext output (Figure 12.9b).

SP 800-38C defines the authentication/encryption process as follows.

1. Apply the formatting function to (N, A, P) to produce the blocks $B_0, B_1, \dots, B_r$.
2. Set $Y_0 = E(K, B_0)$.

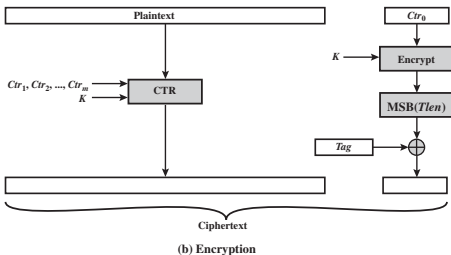
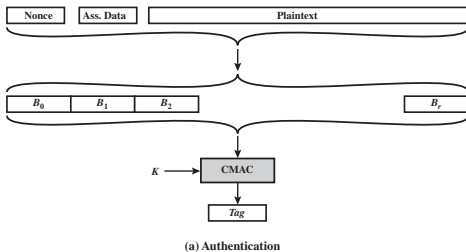


Figure 12.9 Counter with Cipher Block Chaining-Message Authentication Code (CCM)

3. For $i = 1$ to r , do $Y_i = E(K, (B_i \oplus Y_{i-1}))$.
4. Set $T = \text{MSB}_{Tlen}(Y_r)$.
5. Apply the counter generation function to generate the counter blocks $Ctr_0, Ctr_1, \dots, Ctr_m$, where $m = \lceil \text{Plen}/128 \rceil$.
6. For $j = 0$ to m , do $S_j = E(K, Ctr_j)$.
7. Set $S = S_1 \parallel S_2 \parallel \dots \parallel S_m$.
8. Return $C = (P \oplus \text{MSB}_{Plen}(S)) \parallel (T \oplus \text{MSB}_{Tlen}(S_0))$.

For decryption and verification, the recipient requires the following input: the ciphertext C , the nonce N , the associated data A , the key K , and the initial counter Ctr_0 . The steps are as follows.

1. If $Clen \leq Tlen$, then return INVALID.
2. Apply the counter generation function to generate the counter blocks $Ctr_0, Ctr_1, \dots, Ctr_m$, where $m = \lceil Clen/128 \rceil$.
3. For $j = 0$ to m , do $S_j = E(K, Ctr_j)$.
4. Set $S = S_1 \parallel S_2 \parallel \dots \parallel S_m$.
5. Set $P = MSB_{Clen-Tlen}(C) \oplus MSB_{Clen-Tlen}(S)$.
6. Set $T = LSB_{Tlen}(C) \oplus MSB_{Tlen}(S_0)$.
7. Apply the formatting function to (N, A, P) to produce the blocks $B_0, B_1, \dots, B_r$.
8. Set $Y_0 = E(K, B_0)$.
9. For $i = 1$ to r , do $Y_i = E(K, (B_i \oplus Y_{i-1}))$.
10. If $T \neq MSB_{Tlen}(Y_r)$, then return INVALID, else return P .

CCM is a relatively complex algorithm. Note that it requires two complete passes through the plaintext, once to generate the MAC value, and once for encryption. Further, the details of the specification require a tradeoff between the length of the nonce and the length of the tag, which is an unnecessary restriction. Also note that the encryption key is used twice with the CTR encryption mode: once to generate the tag and once to encrypt the plaintext plus tag. Whether these complexities add to the security of the algorithm is not clear. In any case, two analyses of the algorithm ([JONS02] and [ROGA03]) conclude that CCM provides a high level of security.

Galois/Counter Mode

The GCM mode of operation, standardized by NIST in NIST SP 800-38D, is designed to be parallelizable so that it can provide high throughput with low cost and low latency. In essence, the message is encrypted in variant of CTR mode. The resulting ciphertext is multiplied with key material and message length information over $GF(2^{128})$ to generate the authenticator tag. The standard also specifies a mode of operation that supplies the MAC only, known as GMAC.

The GCM mode makes use of two functions: GHASH, which is a keyed hash function, and GCTR, which is essentially the CTR mode with the counters determined by a simple increment by one operation.

GHASH $_H(X)$ takes an input the hash key H and a bit string X such that $\text{len}(X) = 128m$ bits for some positive integer m and produces a 128-bit MAC value. The function may be specified as follows (Figure 12.10a).

1. Let $X_1, X_2, \dots, X_{m-1}, X_m$ denote the unique sequence of blocks such that $X = X_1 \parallel X_2 \parallel \dots \parallel X_{m-1} \parallel X_m$.
2. Let Y_0 be a block of 128 zeros, designated as 0^{128} .
3. For $i = 1, \dots, m$, let $Y_i = (Y_{i-1} \oplus X_i) \cdot H$, where $\cdot$ designates multiplication in $GF(2^{128})$.
4. Return Y_m .

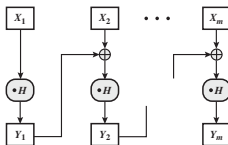
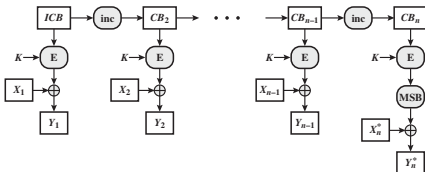
(a) $\text{GHASH}_H(X_1 \parallel X_2 \parallel \dots \parallel X_m) = Y_m$ (b) $\text{GCTR}_K(\text{ICB}, X_1 \parallel X_2 \parallel \dots \parallel X_n^*) = Y_n^*$

Figure 12.10 GCM Authentication and Encryption Functions

The $\text{GHASH}_H(X)$ function can be expressed as

$$(X_1 \cdot H^m) \oplus (X_2 \cdot H^{m-1}) \oplus \dots \oplus (X_{m-1} \cdot H^2) \oplus (X_m \cdot H)$$

This formulation has desirable performance implications. If the same hash key is to be used to authenticate multiple messages, then the values $H^2, H^3, \dots$ can be precalculated one time for use with each message to be authenticated. Then, the blocks of the data to be authenticated ($X_1, X_2, \dots, X_m$) can be processed in parallel, because the computations are independent of one another.

$\text{GCTR}_K(\text{ICB}, X)$ takes as input a secret key K and a bit string X arbitrary length and returns a ciphertext Y of bit length $\text{len}(X)$. The function may be specified as follows (Figure 12.10b).

1. If X is the empty string, then return the empty string as Y .
2. Let $n = \lceil (\text{len}(X)/128) \rceil$. That is, n is the smallest integer greater than or equal to $\text{len}(X)/128$.

3. Let $X_1, X_2, \dots, X_{n-1}, X_n^*$ denote the unique sequence of bit strings such that

$$X = X_1 \parallel X_2 \parallel \dots \parallel X_{n-1} \parallel X_n^*;$$

$X_1, X_2, \dots, X_{n-1}$ are complete 128-bit blocks.

4. Let $CB_1 = ICB$.

5. For, $i = 2$ to n let $CB_i = \text{inc}_{32}(CB_{i-1})$, where the $\text{inc}_{32}(S)$ function increments the rightmost 32 bits of S by 1 mod 2^{32} , and the remaining bits are unchanged.

6. For $i = 1$ to $n - 1$, do $Y_i = X_i \oplus E(K, CB_i)$.

7. Let $Y_n^* = X_n^* \oplus \text{MSB}_{\text{len}(X_n^*)}(E(K, CB_n))$.

8. Let $X = X_1 \parallel X_2 \parallel \dots \parallel X_{n-1} \parallel Y_n^*$

9. Return Y .

Note that the counter values can be quickly generated and that the encryption operations can be performed in parallel.

We can now define the overall authenticated encryption function (Figure 12.11). The input consists of a secret key K , an initialization vector IV , a plaintext P , and

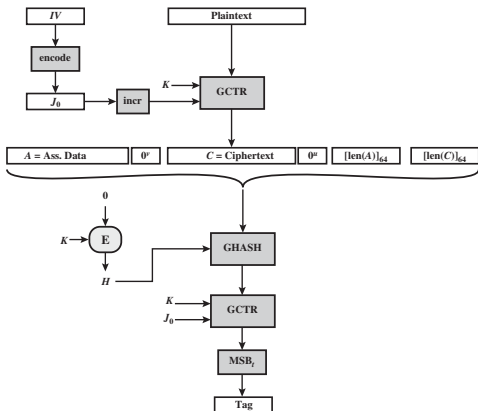


Figure 12.11 Galois Counter— Message Authentication Code (GCM)

additional authenticated data A . The notation $[x]_s$ means the s -bit binary representation of the nonnegative integer x . The steps are as follows.

1. Let $H = E(K, 0^{128})$.
2. Define a block, J_0 , as
 If $\text{len}(IV) = 96$, then let $J_0 = IV \parallel 0^{31} \parallel 1$.
 If $\text{len}(IV) \neq 96$, then let $s = 128 \lceil \text{len}(IV)/128 \rceil - \text{len}(IV)$, and let
 $J_0 = \text{GHASH}_H(IV \parallel 0^{s+64} \parallel [\text{len}(IV)]_{64})$.
3. Let $C = \text{GCTR}_K(\text{inc}_{32}(J_0), P)$.
4. Let $u = 128 \lceil \text{len}(C)/128 \rceil - \text{len}(C)$ and let $v = 128 \lceil \text{len}(A)/128 \rceil - \text{len}(A)$.
5. Define a block, S , as
 $S = \text{GHASH}_H(A \parallel 0^v \parallel C \parallel 0^u \parallel [\text{len}(A)]_{64} \parallel [\text{len}(C)]_{64})$
6. Let $T = \text{MSB}_t(\text{GCTR}_K(J_0, S))$, where t is the supported tag length.
7. Return (C, T) .

In step 1, the hash key is generated by encrypting a block of all zeros with the secret key K . In step 2, the pre-counter block (J_0) is generated from the IV . In particular, when the length of the IV is 96 bits, then the padding string $0^{31} \parallel 1$ is appended to the IV to form the pre-counter block. Otherwise, the IV is padded with the minimum number of 0 bits, possibly none, so that the length of the resulting string is a multiple of 128 bits (the block size); this string in turn is appended with 64 additional 0 bits, followed by the 64-bit representation of the length of the IV , and the GHASH function is applied to the resulting string to form the pre-counter block.

Thus, GCM is based on the CTR mode of operation and adds a MAC that authenticates both the message and additional data that requires only authentication. The function that computes the hash uses only multiplication in a Galois field. This choice was made because the operation of multiplication is easy to perform within a Galois field and is easily implemented in hardware [MCGR05].

[MCGR04] examines the available block cipher modes of operation and shows that a CTR-based authenticated encryption approach is the most efficient mode of operation for high-speed packet networks. The paper further demonstrates that GCM meets a high level of security requirements.

12.8 PSEUDORANDOM NUMBER GENERATION USING HASH FUNCTIONS AND MACS

The essential elements of any pseudorandom number generator (PRNG) are a seed value and a deterministic algorithm for generating a stream of pseudorandom bits. If the algorithm is used as a pseudorandom function (PRF) to produce a required value, such as a session key, then the seed should only be known to the user of the PRF. If the algorithm is used to produce a stream encryption function, then the seed has the role of a secret key that must be known to the sender and the receiver.

We noted in Chapters 7 and 10 that, because an encryption algorithm produces an apparently random output, it can serve as the basis of a (PRNG). Similarly, a hash function or MAC produces apparently random output and can be used to build a PRNG. Both ISO standard 18031 (*Random Bit Generation*) and NIST SP 800-90 (*Recommendation for Random Number Generation Using Deterministic Random Bit Generators*) define an approach for random number generation using a cryptographic hash function. SP 800-90 also defines a random number generator based on HMAC. We look at these two approaches in turn.

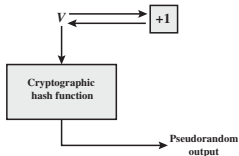
PRNG Based on Hash Function

Figure 12.12a shows the basic strategy for a hash-based PRNG specified in SP 800-90 and ISO 18031. The algorithm takes as input:

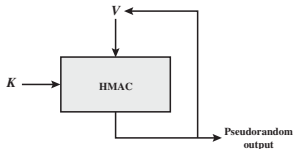
V = seed

$seedlen$ = bit length of $V \geq k + 64$, where k is a desired security level expressed in bits

n = desired number of output bits



(a) PRNG using cryptographic hash function



(b) PRNG using HMAC

Figure 12.12 Basic Structure of Hash-Based PRNGs (SP 800-90)

The algorithm uses the cryptographic hash function H with an hash value output of *outlen* bits. The basic operation of the algorithm is

```

 $m = \lceil n/\text{outlen} \rceil$ 
data =  $V$ 
 $W =$  the null string
For  $i = 1$  to  $m$ 
     $w_i = H(\text{data})$ 
     $W = W \parallel w_i$ 
     $\text{data} = (\text{data} + 1) \bmod 2^{\text{seedlen}}$ 
Return leftmost  $n$  bits of  $W$ 

```

Thus, the pseudorandom bit stream is $w_1 \parallel w_2 \parallel \dots \parallel w_m$ with the final block truncated if required.

The SP 800-90 specification also provides for periodically updating V to enhance security. The specification also indicates that there are no known or suspected weaknesses in the hash-based approach for a strong cryptographic hash algorithm, such as SHA-2.

PRNG Based on MAC Function

Although there are no known or suspected weaknesses in the use of a cryptographic hash function for a PRNG in the manner of Figure 12.12a, a higher degree of confidence can be achieved by using a MAC. Almost invariably, HMAC is used for constructing a MAC-based PRNG. This is because HMAC is a widely used standardized MAC function and is implemented in many protocols and applications. As SP 800-90 points out, the disadvantage of this approach compared to the hash-based approach is that the execution time is twice as long, because HMAC involves two executions of the underlying hash function for each output block. The advantage of the HMAC approach is that it provides a greater degree of confidence in its security, compared to a pure hash-based approach.

For the MAC-based approach, there are two inputs: a key K and a seed V . In effect, the combination of K and V form the overall seed for the PRNG specified in SP 800-90. Figure 12.12b shows the basic structure of the PRNG mechanism, and the leftmost column of Figure 12.13 shows the logic. Note that the key remains the same for each block of output, and the data input for each block is equal to the tag output of the previous block. The SP 800-90 specification also provides for periodically updating K and V to enhance security.

It is instructive to compare the SP 800-90 recommendation with the use of HMAC for a PRNG in some applications, and this is shown in Figure 12.13. For the IEEE 802.11i wireless LAN security standard (Chapter 17), the data input consists of the seed concatenated with a counter. The counter is incremented for each block w_i of output. This approach would seem to offer enhanced security compared to the SP 800-90 approach. Consider that for SP 800-90, the data input for output block w_i is just the output w_{i-1} of the previous execution of HMAC. Thus, an opponent who is able to observe the pseudorandom output knows both the input and output of

$m = \lceil n/\text{outlen} \rceil$ $w_0 = V$ $W = \text{the null string}$ For $i = 1$ to m $w_i = \text{MAC}(K, w_{i-1})$ $W = W \parallel w_i$ Return leftmost n bits of W	$m = \lceil n/\text{outlen} \rceil$ $W = \text{the null string}$ For $i = 1$ to m $w_i = \text{MAC}(K, (V \parallel i))$ $W = W \parallel w_i$ Return leftmost n bits of W	$m = \lceil n/\text{outlen} \rceil$ $A(0) = V$ $W = \text{the null string}$ For $i = 1$ to m $A(i) = \text{MAC}(K, A(i-1))$ $w_i = \text{MAC}(K, (A(i) \parallel V))$ $W = W \parallel w_i$ Return leftmost n bits of W
NIST SP 800-90	IEEE 802.11i	TLS/WTLS

Figure 12.13 Three PRNGs Based on HMAC

HMAC. Even so, with the assumption that HMAC is secure, knowledge of the input and output should not be sufficient to recover K and hence not sufficient to predict future pseudorandom bits.

The approach taken by the Transport Layer Security protocol (Chapter 16) and the Wireless Transport Layer Security Protocol (Chapter 17) involves invoking HMAC twice for each block of output w_i . As with IEEE 802.11, this is done in such a way that the output does not yield direct information about the input. The double use of HMAC doubles the execution burden and would seem to be security overkill.

12.9 RECOMMENDED READING AND WEB SITE

[JUN85] and [JUN87] provide a good background on message authentication with a focus on cryptographic MACs and hash functions. Overviews of HMAC can be found in [BELL96a] and [BELL96b].

BELL96a Bellare, M.; Canetti, R.; and Krawczyk, H. “Keying Hash Functions for Message Authentication.” *Proceedings, CRYPTO '96*, August 1996; published by Springer-Verlag. An expanded version is available at <http://www-cse.ucsd.edu/users/mihir>.

BELL96b Bellare, M.; Canetti, R.; and Krawczyk, H. “The HMAC Construction.” *CryptoBytes*, Spring 1996.

JUN85 Jueneman, R.; Matyas, S.; and Meyer, C. “Message Authentication.” *IEEE Communications Magazine*, September 1988.

JUN87 Jueneman, R. “Electronic Document Authentication.” *IEEE Network Magazine*, April 1987.



Recommended Web Site:

- **Block cipher modes of operation:** NIST page with full information on CMAC, CCM, and GCM.

12.10 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

Key Terms

authenticator Cipher-Based Message Authentication Code (CMAC) CMAC Counter with Cipher Block Chaining-Message Authentication Code (CCM)	cryptographic checksum cryptographic hash function Data Authentication Algorithm (DAA) Galois/Counter Mode (GCM) HMAC	message authentication message authentication code (MAC)
---	--	--

Review Questions

- 12.1 What types of attacks are addressed by message authentication?
- 12.2 What two levels of functionality comprise a message authentication or digital signature mechanism?
- 12.3 What are some approaches to producing message authentication?
- 12.4 When a combination of symmetric encryption and an error control code is used for message authentication, in what order must the two functions be performed?
- 12.5 What is a message authentication code?
- 12.6 What is the difference between a message authentication code and a one-way hash function?
- 12.7 In what ways can a hash value be secured so as to provide message authentication?
- 12.8 Is it necessary to recover the secret key in order to attack a MAC algorithm?
- 12.9 What changes in HMAC are required in order to replace one underlying hash function with another?

Problems

- 12.1 If F is an error-detection function, either internal or external use (Figure 12.2) will provide error-detection capability. If any bit of the transmitted message is altered, this will be reflected in a mismatch of the received FCS and the calculated FCS, whether the FCS function is performed inside or outside the encryption function. Some codes also provide an error-correction capability. Depending on the nature of the function,

- if one or a small number of bits is altered in transit, the error-correction code contains sufficient redundant information to determine the errored bit or bits and correct them. Clearly, an error-correction code will provide error correction capability when used external to the encryption function. Will it also provide this capability if used internal to the encryption function?
- 12.2 The data authentication algorithm, described in Section 12.6, can be defined as using the cipher block chaining (CBC) mode of operation of DES with an initialization vector of zero (Figure 12.7). Show that the same result can be produced using the cipher feedback mode.
- 12.3 At the beginning of Section 12.6, it was noted that given the CBC MAC of a one-block message X , say $T = \text{MAC}(K, X)$, the adversary immediately knows the CBC MAC for the two-block message $X \parallel (X \oplus T)$ since this is once again T . Justify this statement.
- 12.4 In this problem, we demonstrate that for CMAC, a variant that XORs the second key after applying the final encryption doesn't work. Let us consider this for the case of the message being an integer multiple of the block size. Then, the variant can be expressed as $\text{VMAC}(K, M) = \text{CBC}(K, M) \oplus K_1$. Now suppose an adversary is able to ask for the MACs of three messages: the message $\mathbf{0} = 0^n$, where n is the cipher block size; the message $\mathbf{1} = 1^n$; and the message $\mathbf{1} \parallel \mathbf{0}$. As a result of these three queries, the adversary gets $T_0 = \text{CBC}(K, \mathbf{0}) \oplus K_1$; $T_1 = \text{CBC}(K, \mathbf{1}) \oplus K_1$ and $T_2 = \text{CBC}(K, [\text{CBC}(K, \mathbf{1})]) \oplus K_1$. Show that the adversary can compute the correct MAC for the (unqueried) message $\mathbf{0} \parallel (T_0 \oplus T_1)$.
- 12.5 In the discussion of subkey generation in CMAC, it states that the block cipher is applied to the block that consists entirely of 0 bits. The first subkey is derived from the resulting string by a left shift of one bit and, conditionally, by XORing a constant that depends on the block size. The second subkey is derived in the same manner from the first subkey.
- What constants are needed for block sizes of 64 and 128 bits?
 - Explain how the left shift and XOR accomplishes the desired result.
- 12.6 Section 12.6 listed three general approaches to authenticated encryption: MtE, EtM, and E&M.
- Which approach is used by CCM?
 - Which approach is used by GCM?
- 12.7 Show that the GHASH function calculates
- $$(X_1 \cdot H^m) \oplus (X_2 \cdot H^{m-1}) \oplus \cdots \oplus (X_{m-1} \cdot H^2) \oplus (X_m \cdot H)$$
- 12.8 Draw a figure similar to Figure 12.11 that shows authenticated decryption.
- 12.9 Alice wants to send a single bit of information (a yes or a no) to Bob by means of a word of length 2. Alice and Bob have four possible keys available to perform message authentication. The following matrix shows the 2-bit word sent for each message under each key:

Key	Message	
	0	1
1	00	01
2	10	00
3	01	11
4	11	10

- The preceding matrix is in a useful form for Alice. Construct a matrix with the same information that would be more useful for Bob.
- What is the probability that someone else can successfully impersonate Alice?
- What is the probability that someone can replace an intercepted message with another message successfully?

CHAPTER 13

DIGITAL SIGNATURES

13.1 Digital Signatures

- Properties
- Attacks and Forgeries
- Digital Signature Requirements
- Direct Digital Signature

13.2 ElGamal Digital Signature Scheme

13.3 Schnorr Digital Signature Scheme

13.4 Digital Signature Standard

- The DSS Approach
- The Digital Signature Algorithm

13.5 Recommended Reading and Web Site

13.6 Key Terms, Review Questions, and Problems

To guard against the baneful influence exerted by strangers is therefore an elementary dictate of savage prudence. Hence before strangers are allowed to enter a district, or at least before they are permitted to mingle freely with the inhabitants, certain ceremonies are often performed by the natives of the country for the purpose of disarming the strangers of their magical powers, or of disinfecting, so to speak, the tainted atmosphere by which they are supposed to be surrounded.

—*The Golden Bough*, Sir James George Frazer

KEY POINTS

- ◆ A digital signature is an authentication mechanism that enables the creator of a message to attach a code that acts as a signature. Typically the signature is formed by taking the hash of the message and encrypting the message with the creator's private key. The signature guarantees the source and integrity of the message.
- ◆ The digital signature standard (DSS) is an NIST standard that uses the secure hash algorithm (SHA).

The most important development from the work on public-key cryptography is the digital signature. The digital signature provides a set of security capabilities that would be difficult to implement in any other way.

Figure 13.1 is a generic model of the process of making and using digital signatures. Bob can sign a message using a digital signature generation algorithm. The inputs to the algorithm are the message and Bob's private key. Any other user, say Alice, can verify the signature using a verification algorithm, whose inputs are the message, the signature, and Bob's public key. In simplified terms, the essence of the digital signature mechanism is shown in Figure 13.2. This repeats the logic shown in Figure 11.3. A worked-out example, using RSA, is available at this book's Web site.

We begin this chapter with an overview of digital signatures. Then, we introduce the Digital Signature Standard (DSS).

13.1 DIGITAL SIGNATURES

Properties

Message authentication protects two parties who exchange messages from any third party. However, it does not protect the two parties against each other. Several forms of dispute between the two are possible.

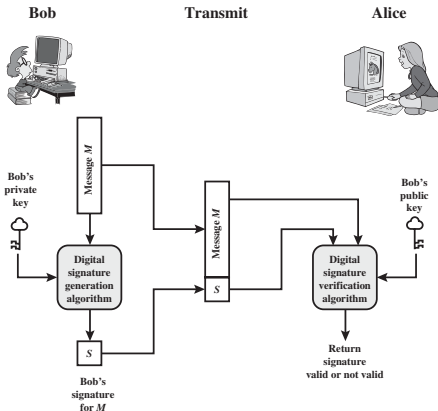


Figure 13.1 Generic Model of Digital Signature Process

For example, suppose that John sends an authenticated message to Mary, using one of the schemes of Figure 12.1. Consider the following disputes that could arise.

1. Mary may forge a different message and claim that it came from John. Mary would simply have to create a message and append an authentication code using the key that John and Mary share.
2. John can deny sending the message. Because it is possible for Mary to forge a message, there is no way to prove that John did in fact send the message.

Both scenarios are of legitimate concern. Here is an example of the first scenario: An electronic funds transfer takes place, and the receiver increases the amount of funds transferred and claims that the larger amount had arrived from the sender. An example of the second scenario is that an electronic mail message contains instructions to a stockbroker for a transaction that subsequently turns out badly. The sender pretends that the message was never sent.

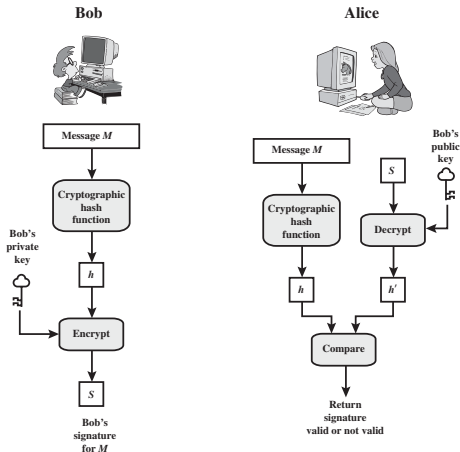


Figure 13.2 Simplified Depiction of Essential Elements of Digital Signature Process

In situations where there is not complete trust between sender and receiver, something more than authentication is needed. The most attractive solution to this problem is the digital signature. The digital signature must have the following properties:

- It must verify the author and the date and time of the signature.
- It must authenticate the contents at the time of the signature.
- It must be verifiable by third parties, to resolve disputes.

Thus, the digital signature function includes the authentication function.

Attacks and Forgeries

[GOLD88] lists the following types of attacks, in order of increasing severity. Here A denotes the user whose signature method is being attacked, and C denotes the attacker.

- **Key-only attack:** C only knows A's public key.
- **Known message attack:** C is given access to a set of messages and their signatures.
- **Generic chosen message attack:** C chooses a list of messages before attempting to break A's signature scheme, independent of A's public key. C then obtains from A valid signatures for the chosen messages. The attack is generic, because it does not depend on A's public key; the same attack is used against everyone.
- **Directed chosen message attack:** Similar to the generic attack, except that the list of messages to be signed is chosen after C knows A's public key but before any signatures are seen.
- **Adaptive chosen message attack:** C is allowed to use A as an "oracle." This means the A may request signatures of messages that depend on previously obtained message-signature pairs.

[GOLD88] then defines success at breaking a signature scheme as an outcome in which C can do any of the following with a non-negligible probability:

- **Total break:** C determines A's private key.
- **Universal forgery:** C finds an efficient signing algorithm that provides an equivalent way of constructing signatures on arbitrary messages.
- **Selective forgery:** C forges a signature for a particular message chosen by C.
- **Existential forgery:** C forges a signature for at least one message. C has no control over the message. Consequently, this forgery may only be a minor nuisance to A.

Digital Signature Requirements

On the basis of the properties and attacks just discussed, we can formulate the following requirements for a digital signature.

- The signature must be a bit pattern that depends on the message being signed.
- The signature must use some information unique to the sender to prevent both forgery and denial.
- It must be relatively easy to produce the digital signature.
- It must be relatively easy to recognize and verify the digital signature.
- It must be computationally infeasible to forge a digital signature, either by constructing a new message for an existing digital signature or by constructing a fraudulent digital signature for a given message.
- It must be practical to retain a copy of the digital signature in storage.

A secure hash function, embedded in a scheme such as that of Figure 13.2, provides a basis for satisfying these requirements. However, care must be taken in the design of the details of the scheme.

Direct Digital Signature

The term **direct digital signature** refers to a digital signature scheme that involves only the communicating parties (source, destination). It is assumed that the destination knows the public key of the source.

Confidentiality can be provided by encrypting the entire message plus signature with a shared secret key (symmetric encryption). Note that it is important to perform the signature function first and then an outer confidentiality function. In case of dispute, some third party must view the message and its signature. If the signature is calculated on an encrypted message, then the third party also needs access to the decryption key to read the original message. However, if the signature is the inner operation, then the recipient can store the plaintext message and its signature for later use in dispute resolution.

The validity of the scheme just described depends on the security of the sender's private key. If a sender later wishes to deny sending a particular message, the sender can claim that the private key was lost or stolen and that someone else forged his or her signature. Administrative controls relating to the security of private keys can be employed to thwart or at least weaken this ploy, but the threat is still there, at least to some degree. One example is to require every signed message to include a timestamp (date and time) and to require prompt reporting of compromised keys to a central authority.

Another threat is that some private key might actually be stolen from X at time T. The opponent can then send a message signed with X's signature and stamped with a time before or equal to T.

The universally accepted technique for dealing with these threats is the use of a digital certificate and certificate authorities. We defer a discussion of this topic until Chapter 14, and focus in this chapter on digital signature algorithms.

13.2 ELGAMAL DIGITAL SIGNATURE SCHEME

Before examining the NIST Digital Signature standard, it will be helpful to understand the ElGamal and Schnorr signature schemes. Recall from Chapter 10, that the ElGamal encryption scheme is designed to enable encryption by a user's public key with decryption by the user's private key. The ElGamal signature scheme involves the use of the private key for encryption and the public key for decryption [ELGA84, ELGA85].

Before proceeding, we need a result from number theory. Recall from Chapter 8 that for a prime number q , if α is a primitive root of q , then

$$\alpha, \alpha^2, \dots, \alpha^{q-1}$$

are distinct (mod q). It can be shown that, if α is a primitive root of q , then

1. For any integer m , $\alpha^m = 1 \pmod{q}$ if and only if $m \equiv 0 \pmod{q-1}$.
2. For any integers i, j , $\alpha^i = \alpha^j \pmod{q}$ if and only if $i \equiv j \pmod{q-1}$.

As with ElGamal encryption, the global elements of **ElGamal digital signature** are a prime number q and α , which is a primitive root of q . User A generates a private/public key pair as follows.

1. Generate a random integer X_A , such that $1 < X_A < q - 1$.
2. Compute $Y_A = \alpha^{X_A} \bmod q$.
3. A's private key is X_A ; A's public key is $\{q, \alpha, Y_A\}$.

To sign a message M , user A first computes the hash $m = H(M)$, such that m is an integer in the range $0 \leq m \leq q - 1$. A then forms a digital signature as follows.

1. Choose a random integer K such that $1 \leq K \leq q - 1$ and $\gcd(K, q - 1) = 1$. That is, K is relatively prime to $q - 1$.
2. Compute $S_1 = \alpha^K \bmod q$. Note that this is the same as the computation of C_1 for ElGamal encryption.
3. Compute $K^{-1} \bmod (q - 1)$. That is, compute the inverse of K modulo $q - 1$.
4. Compute $S_2 = K^{-1}(m - X_A S_1) \bmod (q - 1)$.
5. The signature consists of the pair (S_1, S_2) .

Any user B can verify the signature as follows.

1. Compute $V_1 = \alpha^m \bmod q$.
2. Compute $V_2 = (Y_A)^{S_1} (S_2)^{S_2} \bmod q$.

The signature is valid if $V_1 = V_2$. Let us demonstrate that this is so. Assume that the equality is true. Then we have

$$\begin{array}{ll}
 \alpha^m \bmod q = (Y_A)^{S_1} (S_2)^{S_2} \bmod q & \text{assume } V_1 = V_2 \\
 \alpha^m \bmod q = \alpha^{X_A S_1} \alpha^{K S_2} \bmod q & \text{substituting for } Y_A \text{ and } S_1 \\
 \alpha^{m - X_A S_1} \bmod q = \alpha^{K S_2} \bmod q & \text{rearranging terms} \\
 m - X_A S_1 = K S_2 \bmod (q - 1) & \text{property of primitive roots} \\
 m - X_A S_1 = K K^{-1} (m - X_A S_1) \bmod (q - 1) & \text{substituting for } S_2
 \end{array}$$

For example, let us start with the prime field $\text{GF}(19)$; that is, $q = 19$. It has primitive roots $\{2, 3, 10, 13, 14, 15\}$, as shown in Table 8.3. We choose $\alpha = 10$.

Alice generates a key pair as follows:

1. Alice chooses $X_A = 16$.
2. Then $Y_A = \alpha^{X_A} \bmod q = 10^{16} \bmod 19 = 4$.
3. Alice's private key is 16; Alice's public key is $\{q, \alpha, Y_A\} = \{19, 10, 4\}$.

Suppose Alice wants to sign a message with hash value $m = 14$.

1. Alice chooses $K = 5$, which is relatively prime to $q - 1 = 18$.
2. $S_1 = \alpha^K \bmod q = 10^5 \bmod 19 = 3$ (see Table 8.3).

3. $K^{-1} \bmod (q - 1) = 5^{-1} \bmod 18 = 11$.
4. $S_2 = K^{-1}(m - X_A S_1) \bmod (q - 1) = 11(14 - (16)(3)) \bmod 18 = -374 \bmod 18 = 4$.

Bob can verify the signature as follows.

1. $V_1 = \alpha^m \bmod q = 10^{14} \bmod 19 = 16$.
2. $V_2 = (Y_A)^{S_1}(S_2)^{S_2} \bmod q = (4^3)(3^4) \bmod 19 = 5184 \bmod 19 = 16$.

Thus, the signature is valid.

13.3 SCHNORR DIGITAL SIGNATURE SCHEME

As with the ElGamal digital signature scheme, the Schnorr signature scheme is based on discrete logarithms [SCHN89, SCHN91]. The Schnorr scheme minimizes the message-dependent amount of computation required to generate a signature. The main work for signature generation does not depend on the message and can be done during the idle time of the processor. The message-dependent part of the signature generation requires multiplying a $2n$ -bit integer with an n -bit integer.

The scheme is based on using a prime modulus p , with $p - 1$ having a prime factor q of appropriate size; that is, $p - 1 \equiv (\bmod q)$. Typically, we use $p \approx 2^{1024}$ and $q \approx 2^{160}$. Thus, p is a 1024-bit number, and q is a 160-bit number, which is also the length of the SHA-1 hash value.

The first part of this scheme is the generation of a private/public key pair, which consists of the following steps.

1. Choose primes p and q , such that q is a prime factor of $p - 1$.
2. Choose an integer a , such that $a^q = 1 \bmod p$. The values a , p , and q comprise a global public key that can be common to a group of users.
3. Choose a random integer s with $0 < s < q$. This is the user's private key.
4. Calculate $v = a^{-s} \bmod p$. This is the user's public key.

A user with private key s and public key v generates a signature as follows.

1. Choose a random integer r with $0 < r < q$ and compute $x = a^r \bmod p$. This computation is a preprocessing stage independent of the message M to be signed.
2. Concatenate the message with x and hash the result to compute the value e :

$$e = H(M \parallel x)$$

3. Compute $y = (r + se) \bmod q$. The signature consists of the pair (e, y) .

Any other user can verify the signature as follows.

1. Compute $x' = a^y v^e \bmod p$.
2. Verify that $e = H(M \parallel x')$.

To see that the verification works, observe that

$$x' = a^y v^e = a^y a^{-se} = a^{y-se} = a^r = x \pmod{p}$$

Hence, $H(M \parallel x') = H(M \parallel x)$.

13.4 DIGITAL SIGNATURE STANDARD

The National Institute of Standards and Technology (NIST) has published Federal Information Processing Standard FIPS 186, known as the Digital Signature Standard (DSS). The DSS makes use of the Secure Hash Algorithm (SHA) described in Chapter 12 and presents a new digital signature technique, the **Digital Signature Algorithm (DSA)**. The DSS was originally proposed in 1991 and revised in 1993 in response to public feedback concerning the security of the scheme. There was a further minor revision in 1996. In 2000, an expanded version of the standard was issued as FIPS 186-2, subsequently updated to FIPS 186-3 in 2009. This latest version also incorporates digital signature algorithms based on RSA and on elliptic curve cryptography. In this section, we discuss the original DSS algorithm.

The DSS Approach

The DSS uses an algorithm that is designed to provide only the digital signature function. Unlike RSA, it cannot be used for encryption or key exchange. Nevertheless, it is a public-key technique.

Figure 13.3 contrasts the DSS approach for generating digital signatures to that used with RSA. In the RSA approach, the message to be signed is input to a hash function that produces a secure hash code of fixed length. This hash code is then encrypted using the sender's private key to form the signature. Both the message and the signature are then transmitted. The recipient takes the message and produces a hash code. The recipient also decrypts the signature using the sender's public key. If the calculated hash code matches the decrypted signature, the signature is accepted as valid. Because only the sender knows the private key, only the sender could have produced a valid signature.

The DSS approach also makes use of a hash function. The hash code is provided as input to a signature function along with a random number k generated for this particular signature. The signature function also depends on the sender's private key (PR_A) and a set of parameters known to a group of communicating principals. We can consider this set to constitute a global public key (PU_G).¹ The result is a signature consisting of two components, labeled s and r .

¹It is also possible to allow these additional parameters to vary with each user so that they are a part of a user's public key. In practice, it is more likely that a global public key will be used that is separate from each user's public key.

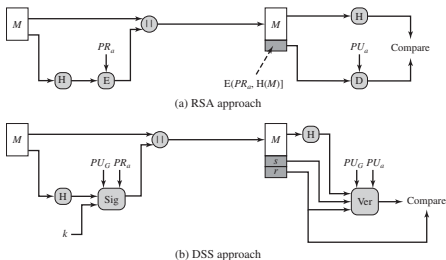


Figure 13.3 Two Approaches to Digital Signatures

At the receiving end, the hash code of the incoming message is generated. This plus the signature is input to a verification function. The verification function also depends on the global public key as well as the sender's public key (PU_a), which is paired with the sender's private key. The output of the verification function is a value that is equal to the signature component r if the signature is valid. The signature function is such that only the sender, with knowledge of the private key, could have produced the valid signature.

We turn now to the details of the algorithm.

The Digital Signature Algorithm

The DSA is based on the difficulty of computing discrete logarithms (see Chapter 8) and is based on schemes originally presented by ElGamal [ELGA85] and Schnorr [SCHN91].

Figure 13.4 summarizes the algorithm. There are three parameters that are public and can be common to a group of users. A 160-bit prime number q is chosen. Next, a prime number p is selected with a length between 512 and 1024 bits such that q divides $(p - 1)$. Finally, g is chosen to be of the form $h^{(p-1)/q} \bmod p$, where h is an integer between 1 and $(p - 1)$ with the restriction that g must be greater than 1.² Thus, the global public-key components of DSA have the same for as in the Schnorr signature scheme.

With these numbers in hand, each user selects a private key and generates a public key. The private key x must be a number from 1 to $(q - 1)$ and should be chosen randomly or pseudorandomly. The public key is calculated from the private key as $y = g^x \bmod p$. The calculation of y given x is relatively straightforward. However, given the public key y , it is believed to be computationally infeasible to determine x , which is the discrete logarithm of y to the base g , $\bmod p$ (see Chapter 8).

²In number-theoretic terms, g is of order $q \bmod p$; see Chapter 8.

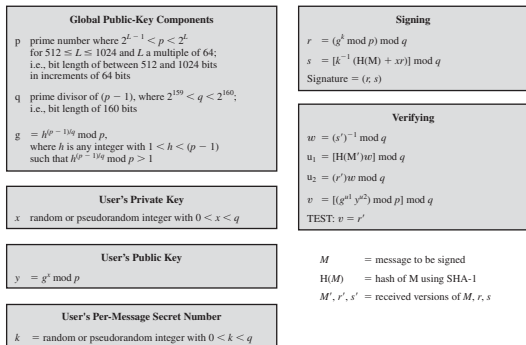


Figure 13.4 The Digital Signature Algorithm (DSA)

To create a signature, a user calculates two quantities, r and s , that are functions of the public key components (p, q, g) , the user's private key (x) , the hash code of the message $H(M)$, and an additional integer k that should be generated randomly or pseudorandomly and be unique for each signing.

At the receiving end, verification is performed using the formulas shown in Figure 13.4. The receiver generates a quantity v that is a function of the public key components, the sender's public key, and the hash code of the incoming message. If this quantity matches the r component of the signature, then the signature is validated.

Figure 13.5 depicts the functions of signing and verifying.

The structure of the algorithm, as revealed in Figure 13.5, is quite interesting. Note that the test at the end is on the value r , which does not depend on the message at all. Instead, r is a function of k and the three global public-key components. The multiplicative inverse of $k \pmod{q}$ is passed to a function that also has as inputs the message hash code and the user's private key. The structure of this function is such that the receiver can recover r using the incoming message and signature, the public key of the user, and the global public key. It is certainly not obvious from Figure 13.4 or Figure 13.5 that such a scheme would work. A proof is provided in Appendix K.

Given the difficulty of taking discrete logarithms, it is infeasible for an opponent to recover k from r or to recover x from s .

Another point worth noting is that the only computationally demanding task in signature generation is the exponential calculation $g^k \bmod p$. Because this value does not depend on the message to be signed, it can be computed ahead of time.

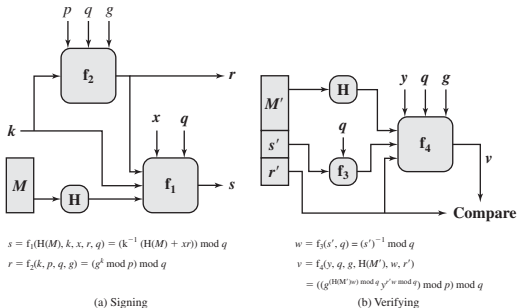


Figure 13.5 DSS Signing and Verifying

Indeed, a user could precalculate a number of values of r to be used to sign documents as needed. The only other somewhat demanding task is the determination of a multiplicative inverse, k^{-1} . Again, a number of these values can be precalculated.

13.5 RECOMMENDED READING AND WEB SITE

[AKL83] is the classic paper on digital signatures and is still highly relevant. A more recent, and excellent, survey is [MITC92].

AKL83 Akl, S. "Digital Signatures: A Tutorial Survey." *Computer*, February 1983.

MITC92 Mitchell, C.; Piper, F.; and Wild, P. "Digital Signatures." In [SIMM92a].



Recommended Web Site:

- **Digital Signatures:** NIST page with information on NIST-approved digital signature options.